EE5141: Introduction to Wireless and Cellular Communication

Simulation Assignment 1 - Report

January - May 2026

Name: Vignesh Natarajkumar
Roll No.: EE23B087

Question 1

1. **Question:** A vehicle is moving with velocity v=30 m/s where the wireless link uses a carrier-frequency of fc=2GHz and a pass-band message bandwidth of 2W=100KHz. Assume the Jakes' PSD.

   1a) Simulate and plot one run of a single fading complex-valued path gain (sampled every Ts=1/2W=10 microsecs), over N=8000 samples, using each of the 3 following fading models:

   - Linear Prediction (Levinson Algorithm) based noise coloring filter. See class notes. Also refer to [4].
   - Smith's model (using FFTs) – see Rappaport's book; use N=8192 here. Also see [4]
   - Modified sum of sinusoids model using [2]. Also see [1] and [3].

   1b) Also plot for each case, the auto-correlation function, as well as the cross-correlation between the real and imaginary values of the path gain. Comment.

   Answer: First, here are some fundamental points that are worth noting for this assignment:

   - For a particular path with a certain long-term channel gain and a delay with respect to some reference, small-scale fading results in a complex channel coefficient at each time instant. These complex random variables form a random process.
   - The complex channel corresponding to a path is the baseband representation of the channel, and the actual carrier frequency has no influence here. It only plays a role in the conversion from speed to Doppler frequency.
   - Realistically, the multiplicative channel will take into account other phenomena such as synchronization issues, carrier offset etc.
   - Due to superposition from multiple surfaces, the complex channel gain can be modeled as a complex Gaussian process in time due to CLT. Based on whether a Line of Sight (LoS) path is present or not, the magnitude of the complex channel gain can be modeled with either the Rayleigh or the Rician distribution, specifying the first-order statistics.
   - Regarding the second-order statistics of this process (and we need only the first and second order statistics for a Gaussian process), the complex channel coefficient of a path is ideally constant (i.e. one sample from the Rayleigh distribution) if the physical environment of the receiver is static. However, relative transceiver motion will lead to a time-varying channel gain and a Doppler spread.
   - How fast the complex channel gain of a single path varies (which is based on the speed/Doppler frequency) determines if the channel is fast fading or slow fading.

Based on the parameters provided in the question, 3 methods to produce a complex random process with a specified Power Spectral Density (PSD) have been implemented, using context obtained from [3, 1, 2, 4]

- **Linear Prediction (Levinson's Algorithm**) models the complex fading channel as an autoregressive process with a chosen lag order. By using the Z-Transform, such a system can be implemented using a filter.

  The optimal coefficients for this AR process will minimize the ensemble-averaged MSE between the prediction and every candidate signal path that has a Jakes' PSD. For an AR process of order p, we need to solve p equations which can be expressed as autocorrelation equations, called Yule-Walker equations. The Levinson-Durbin algorithm helps us solve these p equations with a lower time complexity, and the obtained filter can be applied to a discrete-time complex Gaussian process to get the desired PSD.

  The theoretical idea behind this method is Wold's Representation Theorem, which states that any Wide Sense Stationary (WSS) process (for which the PSD is well-defined) can be modeled using an infinite order AR process. However, finite-order AR pocesses will introduce tracking errors.

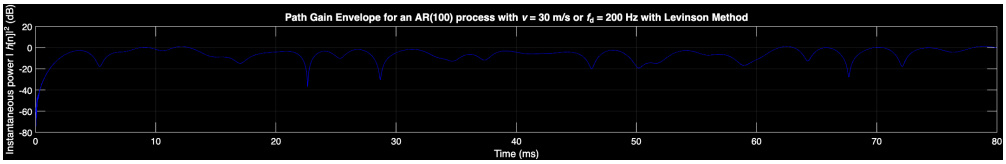

Figure 1: Path Gain Envelope ($|h[n]|^2$ in dB) with $v = 30$ m/s using the Levinson Method.

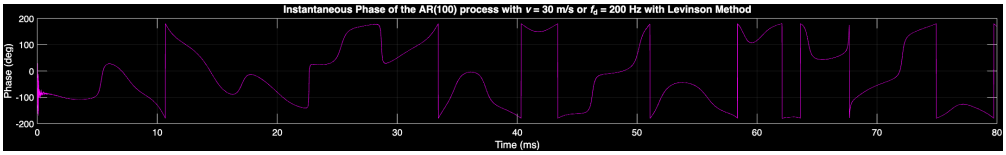

Figure 2: Instantaneous Phase (degrees) with $v = 30$ m/s using the Levinson Method.

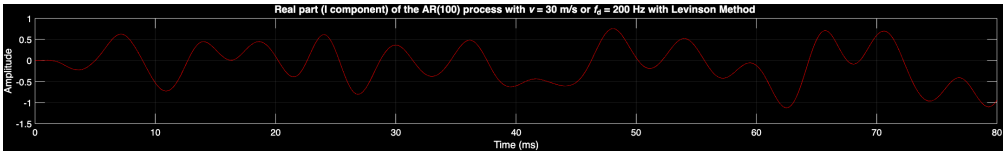

Figure 3: Real part (I component) with $v = 30$ m/s using the Levinson Method.

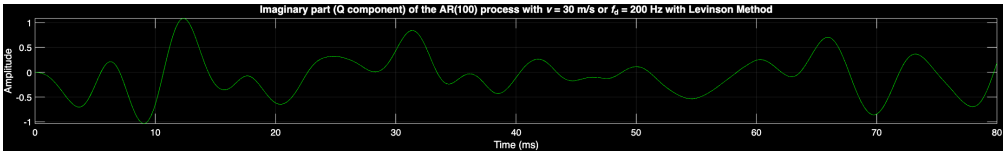

Figure 4: Imaginary part (Q component) with $v = 30$ m/s using the Levinson Method.

- **Smith's Model** uses the property of linear time-invariant (LTI) systems on the PSD of a WSS input:

  This equation describes the relationship between the PSD of a WSS input signal, $S_w(f)$, and the output signal, $S_x(f)$, after passing through a LTI system with a frequency response of $H(f)$.

$$S_x(f) = |H(f)|^2 S_w(f)$$

  Hence, if we are aware of the PSD we want at the output $S_x(f)$, then we can simply design a filter and an input so that the output has a PSD shaped like $S_x(f)$.

  Smith's method generates a discrete complex Gaussian process in time. The PSD of this time-domain process will be a flat line, which means that the required filter $H(f)$ to get the desired $S_{jakes}(f)$ will be a scaled version of $\sqrt{S_{jakes}(f)}$. An IFFT then provides the time-domain fading channel with the desired PSD.

  Instead of producing a Gaussian process in time and running the FFT algorithm, the alternative, as described in [4] is to produce a complex Gaussian process directly in frequency, because Gaussianity is preserved with linear operations, such as FFT.

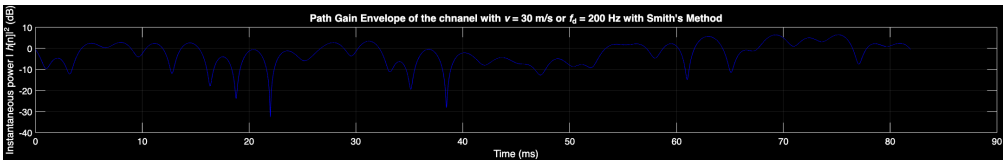

Figure 5: Path Gain Envelope ($|h[n]|^2$ in dB) with $v = 30$ m/s using Smith's Model.

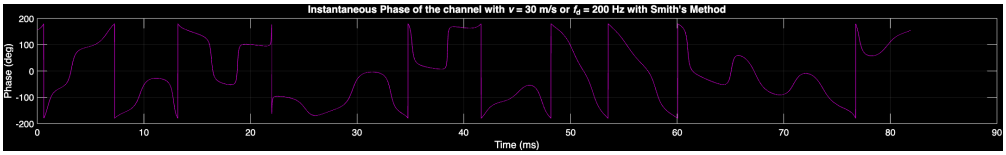

Figure 6: Instantaneous Phase (degrees) with $v = 30$ m/s using Smith's Model.

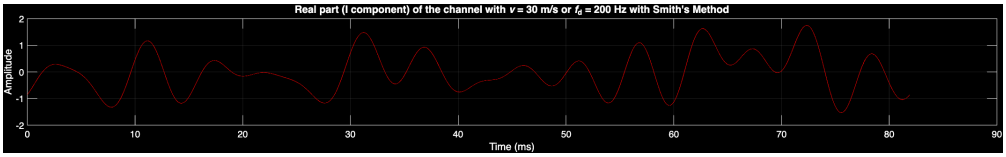

Figure 7: Real part (I component) with $v = 30$ m/s using Smith's Model.

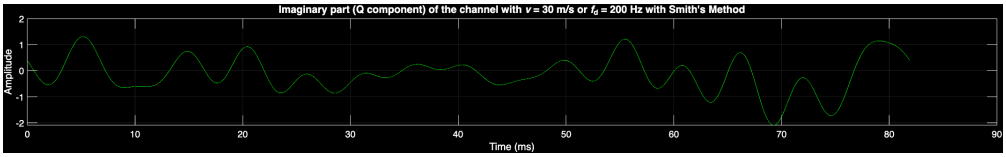

Figure 8: Imaginary part (Q component) with $v = 30$ m/s using Smith's Model.

- **Modified Sum of Sinusoids (SoS) model** uses the physical intuition behind small-scale scattering discussed in class, but with some modifications to ensure that independent channel instances (as are needed for MIMO simulations) are uncorrelated and the second-order statistics are as desired. [7] proposes new Rayleigh fading channel models that have the desired statistical properties even when the number of sinusoids is small.

$$Z(t) = Z_c(t) + jZ_s(t) \tag{3a}$$

$$Z_c(t) = \sqrt{\frac{2}{M}} \sum_{n=1}^{M} \cos(w_d t \cos \alpha_n + \phi_n) \tag{3b}$$

$$Z_s(t) = \sqrt{\frac{2}{M}} \sum_{n=1}^{M} \cos(w_d t \sin \alpha_n + \varphi_n) \tag{3c}$$

with

$$\alpha_n = \frac{2\pi n - \pi + \theta}{4M}, \quad n = 1, 2, \ldots, M \tag{4}$$

where ϕ_n, φ_n and θ are independent and uniformly distributed on $[-\pi, \pi)$ for all n.

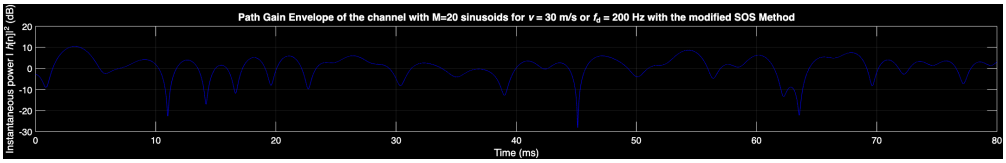

Figure 9: Path Gain Envelope ($|h[n]|^2$ in dB) with $v = 30$ m/s using the SOS Method.

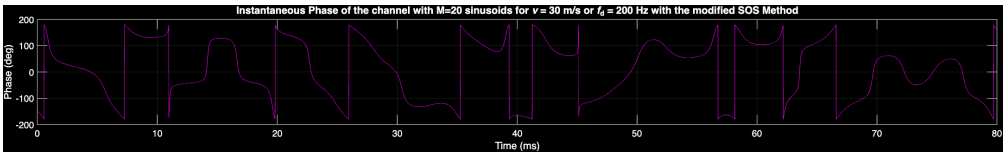

Figure 10: Instantaneous Phase (degrees) with $v = 30$ m/s using the SOS Method.

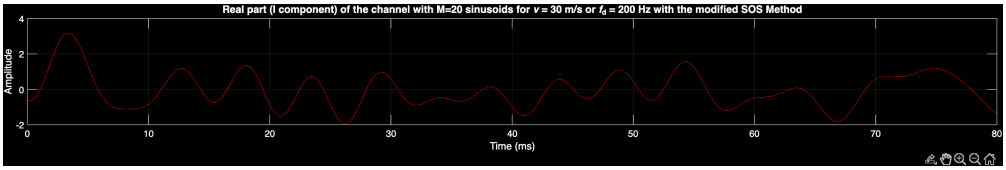

Figure 11: Real part (I component) with $v = 30$ m/s using the SOS Method.

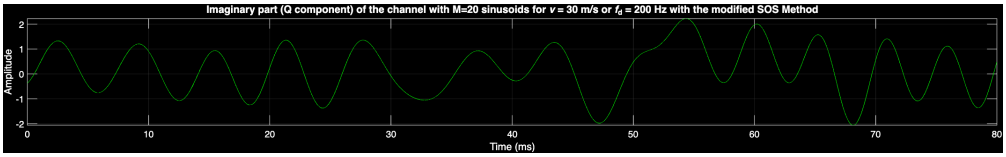

Figure 12: Imaginary part (Q component) with $v = 30$ m/s using the SOS Method.

The below plots show the autocorrelation of 1 complex fading channel, as well as the cross-correlation of the real and imaginary processes of the complex fading channel. The statistical adjustments made by the SoS model help make it the closest to the expected correlation values, measured by the RMS plotted in each plot.

- **Levinson Method**

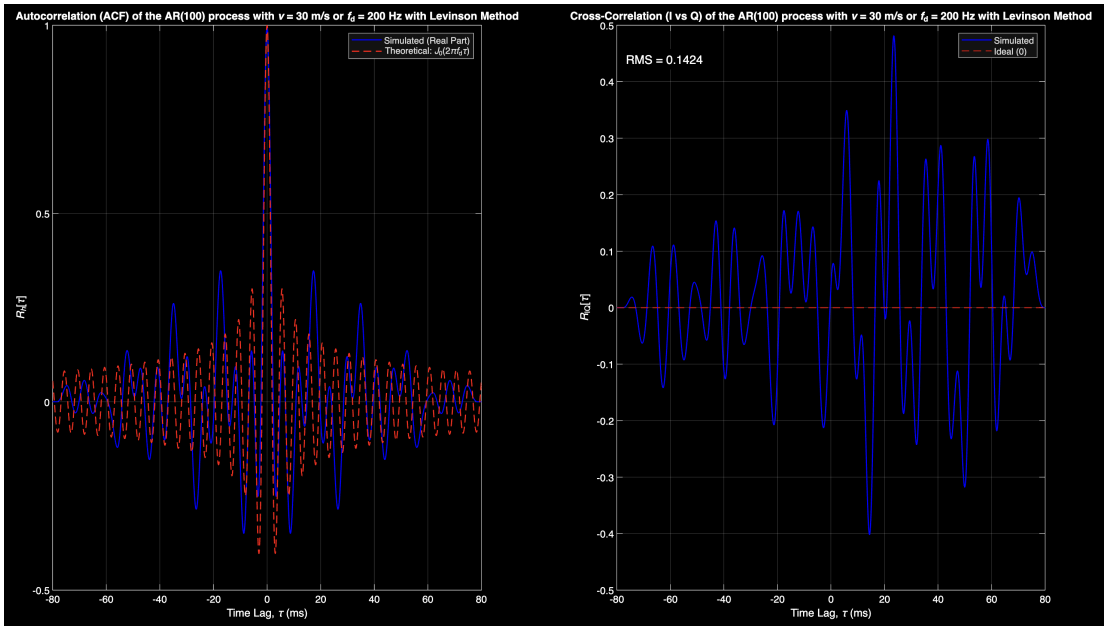

Figure 13: Auto-correlation and cross-correlation of the channel generated with Levinson Method.

- **Smith's Method**

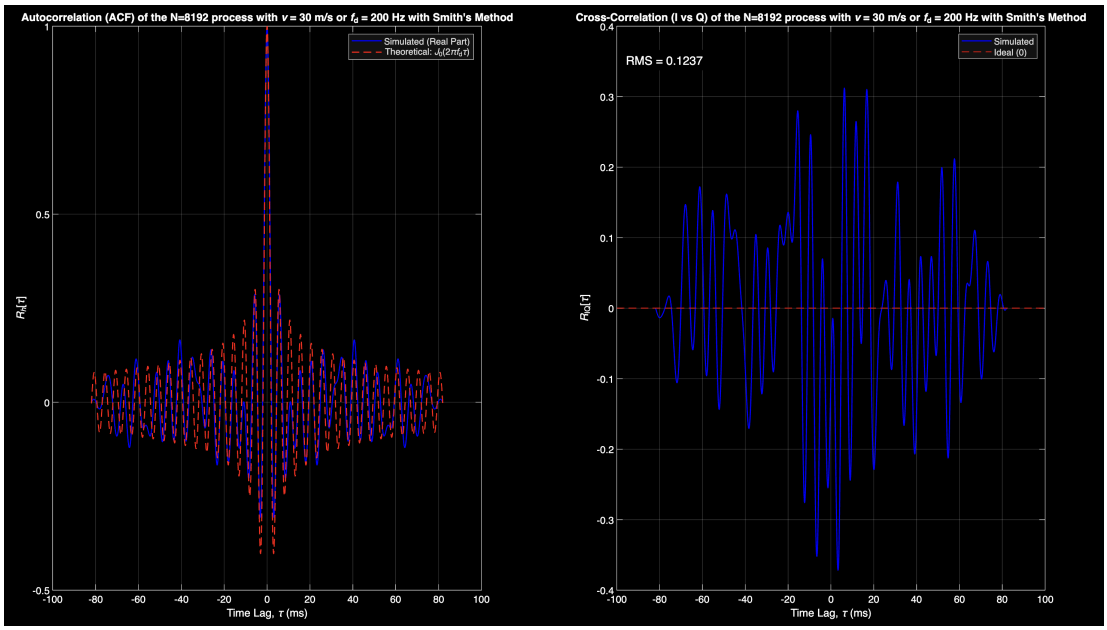

Figure 14: Auto-correlation and cross-correlation of the channel generated with Smith's Model.

- **Modified Sum of Sinusoids (SoS) model**

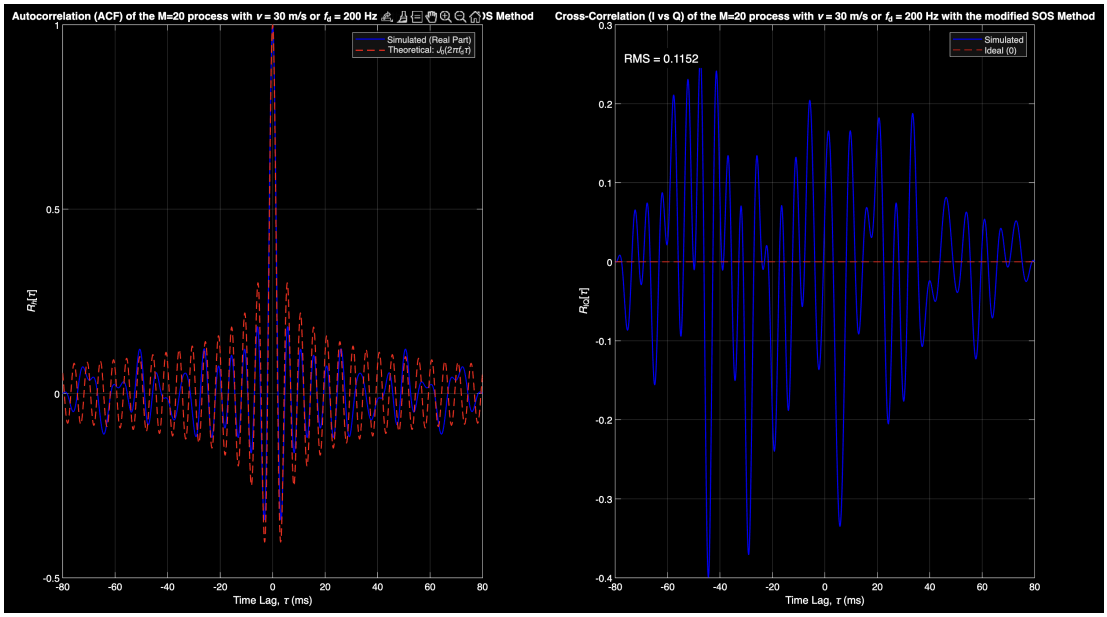

Figure 15: Auto-correlation and cross-correlation of the channel generated with SOS Method.

At this stage, we can make some comments:

- All the 3 methods provide a method to model the time-variation in the channel introduced due to a Doppler spread.
- The Sum of Sinusoids method appears to track the Bessel autocorrelation the best, while minimizing the cross-correlation between the real and imaginary parts of the channel (measured by the RMS value)
- The performance of the Levinson channel tracking method is expected to improve as the order p of the autoregressive process is increased, but there will never be perfect tracking as discussed earlier.
- Another advantage of the modified SoS method is that it remains faithful to many statistical expectations of the channel even when a small number of sinusoidal oscillators M are considered.

It is also interesting to see if the correlation between 2 independent channel instances falls to 0 in the 3 methods, as is expected ideally with MIMO simulations. This is plotted for the 3 methods below:

- **Levinson Method**

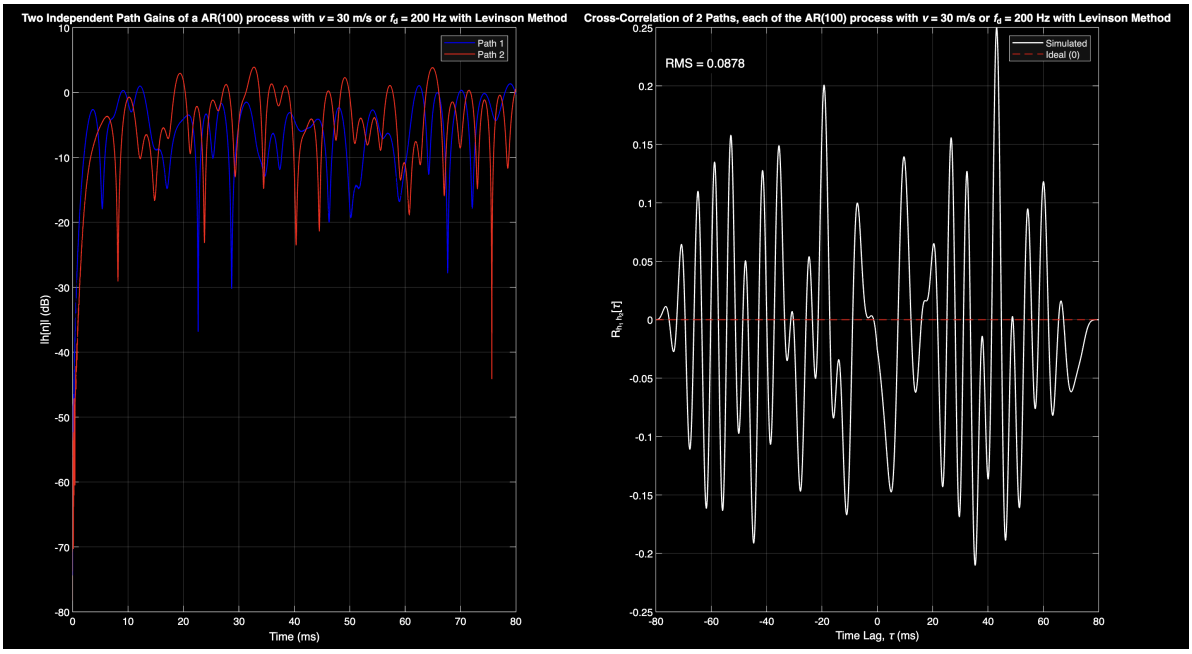

Figure 16: 2 path correlation of the channel generated with Levinson Method.

- **Smith's Method**

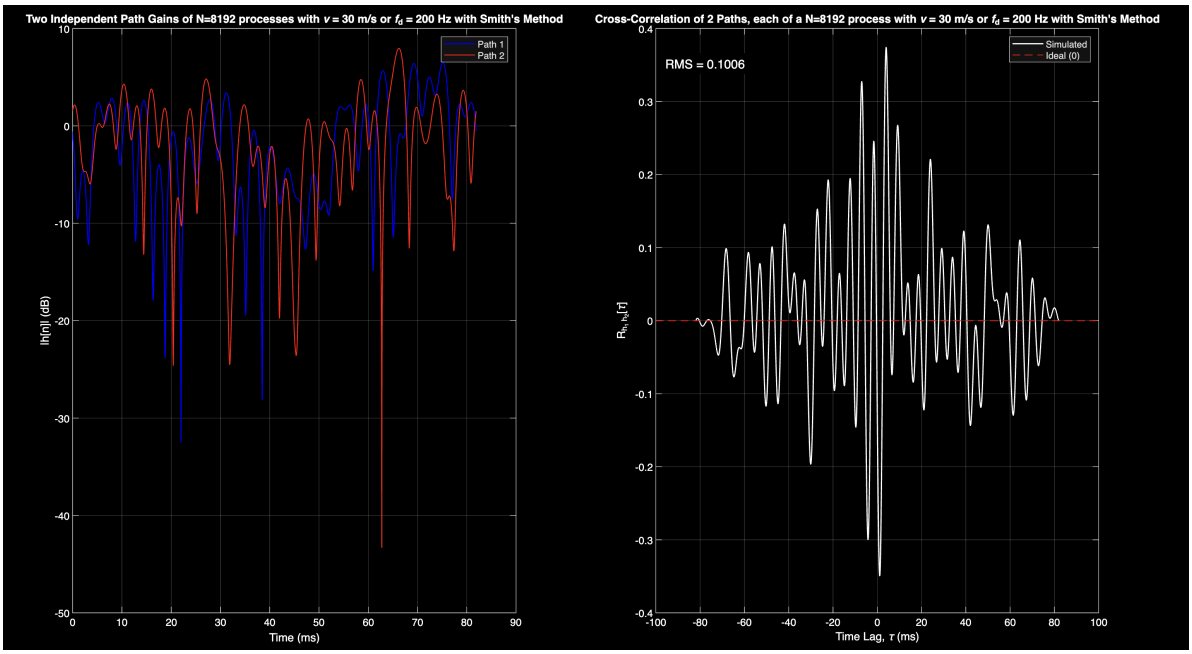

Figure 17: 2 path correlation of the channel generated with Smith's Model.

- **Modified Sum of Sinusoids (SoS) model**

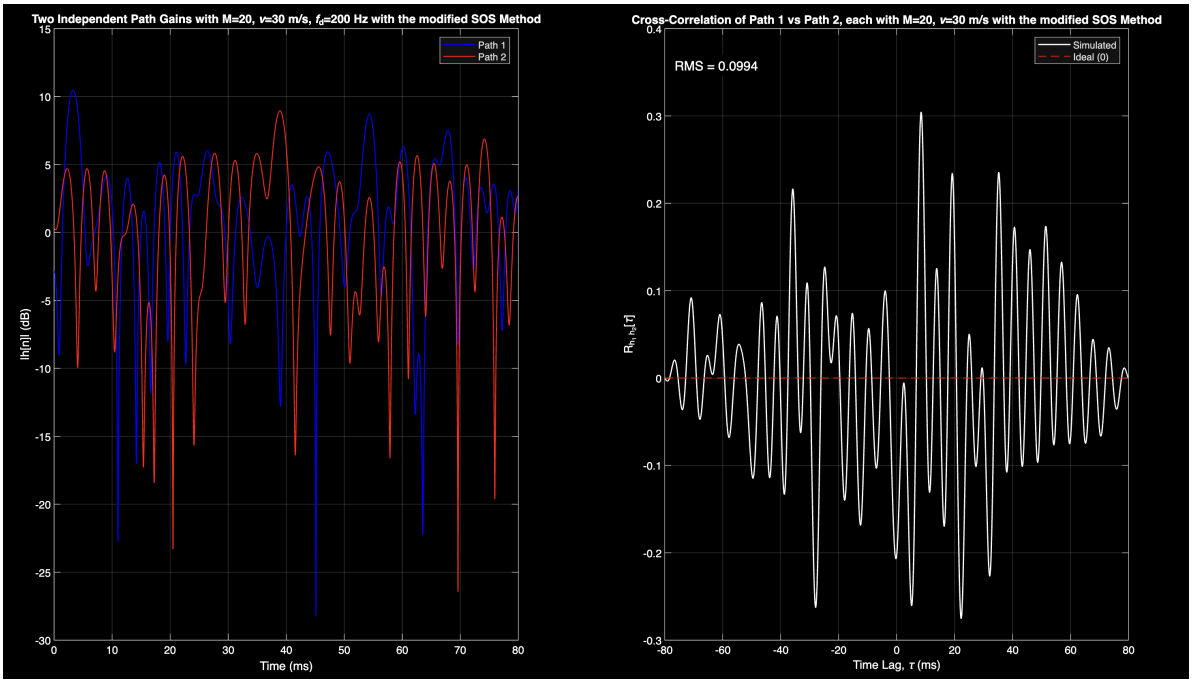

Figure 18: 2 path correlation of the channel generated with SOS Method.

2. **Question:** Repeat 1a. when the velocity is only v=3m/s. Comment.

Answer: When the Doppler frequency increases, the correlation between the channels also reduces since the Doppler spread of the PSD increases, and the PSD becomes a flat line in the limiting case of infinite relative transciever mobility. A flat PSD implies a Dirac delta autocorrelation, which means that the samples are uncorrelated, and the fading process is composed of IID complex Gaussian samples at each time instance, which all follow a magnitude distribution described by either Rayleigh or Rician distributions.

Hence, when we reduce the Doppler frequency, the autocorrelation becomes more time-spread, and the channel samples become more correlated in time. As a result, we would expect the lower velocity of 3 m/s to produce a more smoothly varying channel as compared to the 30 m/s case.

The outputs for the different methods is given here:

- **Levinson method**

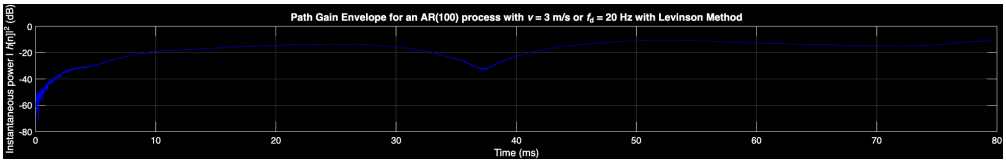

Figure 19: Path Gain Envelope ($|h[n]|^2$ in dB) with $v = 3$ m/s using the Levinson Method.

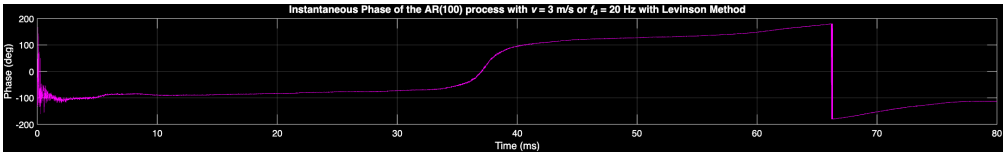

Figure 20: Instantaneous Phase (degrees) with $v = 3$ m/s using the Levinson Method.

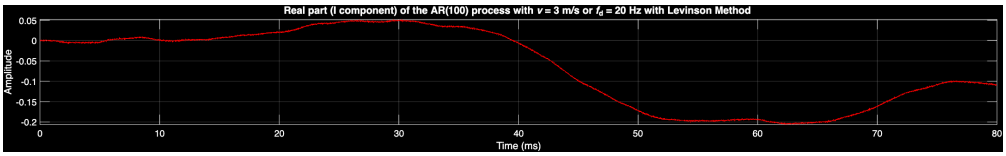

Figure 21: Real part (I component) with $v = 3$ m/s using the Levinson Method.

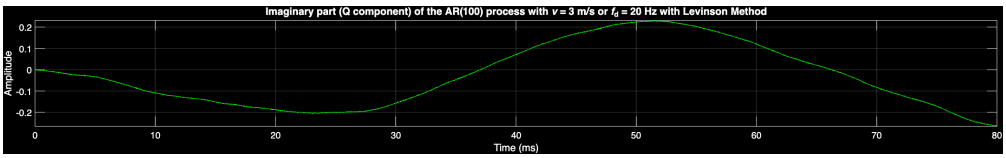

Figure 22: Imaginary part (Q component) with $v = 3$ m/s using the Levinson Method.

- **Smith's method**

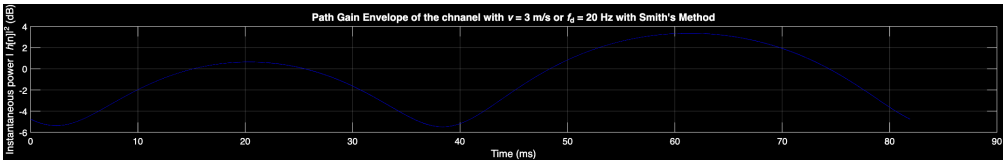

Figure 23: Path Gain Envelope ($|h[n]|^2$ in dB) with $v = 3$ m/s using Smith's Model.

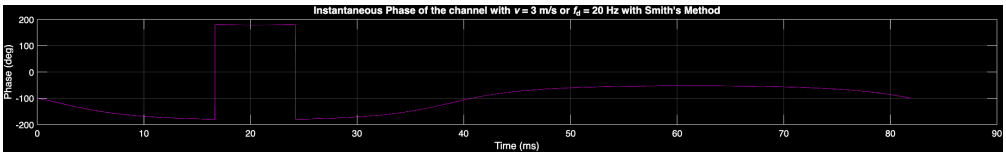

Figure 24: Instantaneous Phase (degrees) with $v = 3$ m/s using Smith's Model.

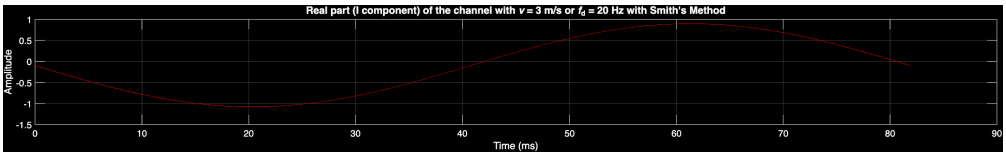

Figure 25: Real part (I component) with $v = 3$ m/s using Smith's Model.

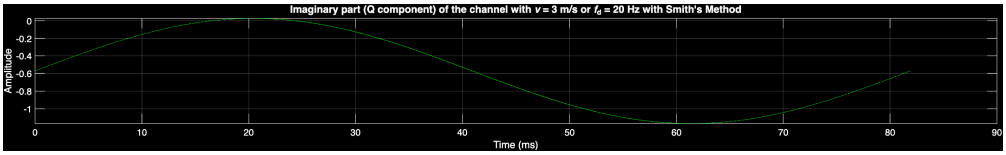

Figure 26: Imaginary part (Q component) with $v = 3$ m/s using Smith's Model.

- **Modified Sum of Sinusoids (SoS) model**

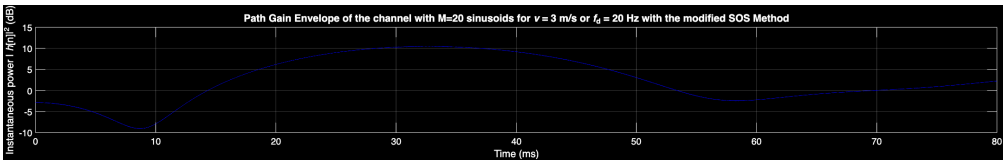

Figure 27: Path Gain Envelope ($|h[n]|^2$ in dB) with $v = 3$ m/s using the SOS Method.

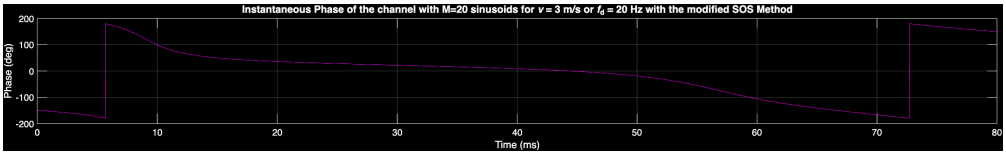

Figure 28: Instantaneous Phase (degrees) with $v = 3$ m/s using the SOS Method.

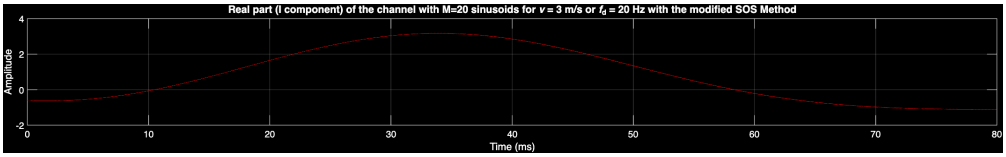

Figure 29: Real part (I component) with $v = 3$ m/s using the SOS Method.

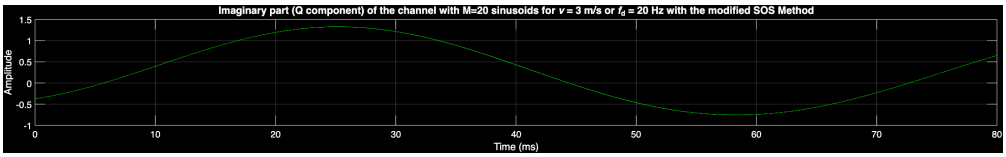

Figure 30: Imaginary part (Q component) with $v = 3$ m/s using the SOS Method.

3. **Question:** Bonus Question [3 marks]: Read Suzuki model in [5] or the flexible approach in [6], and simulate either of them. Can you get the bathtub Jake's PSD as a special case of them? Comment.

Answer: I choose to simulate the flexible approach in [6]. Our models so far assume a 2D uniform Power Azimuth Spectrum (PAS) and the usage of omni-antennas. The PAS captures the combined effects of the base station's antenna pattern and the physical scattering environment.

The paper argues that the Jakes' PSD does not consider the physical environment accurately with non-uniform antenna patterns and non-isotropic scattering. For example, the coherence time may not be small for a large Doppler frequency with directional antenna effects.

Let the unnormalized PAS be denoted as $q(\theta)$, where the azimuth angle $\theta \in (-\pi, \pi]$. The total power of the Non-Line-Of-Sight (NLOS) component associated with the l-th path between the m_t-th Tx antenna and m_r-th Rx antenna is defined as:

$$\frac{P_{m_r,m_t,l}}{K_{m_r,m_t,l}+1} = \int_{-\pi}^{\pi} q(\theta) d\theta \tag{1}$$

where P is the path power and K is the Rician factor.

From this, the normalized PAS, which acts as a Probability Density Function (PDF) of the azimuth angle, is obtained by dividing the unnormalized PAS by the total NLOS power:

$$p(\theta) = \frac{q(\theta)}{\int_{-\pi}^{\pi} q(\theta) d\theta} \tag{2}$$

To flexibly model this, $p(\theta)$ can be written as a mixture of N von Mises distributions, where each characterizes one sub-cluster of scatterers:

$$p(\theta) = \sum_{n=1}^{N} p_n \frac{\exp[\kappa_n \cos(\theta - \phi_n)]}{2\pi I_0(\kappa_n)}, \quad \theta \in (-\pi, \pi] \tag{3}$$

where:

- $I_0(z)$ is the zero-th order modified Bessel function of the first kind.
- ϕ_n is the mean angle of arrival for the n-th sub-cluster.
- $\kappa_n \geq 0$ is the concentration parameter, which controls the angle spread of the n-th sub-cluster.
- p_n represents the fractional power contribution of the n-th sub-cluster, such that $\sum_{n=1}^{N} p_n = 1$.

One thing to note is that the product of two von Mises mixtures (e.g., a directional antenna pattern $G(\theta)$ and a non-isotropic scattering environment) results in another von Mises mixture.

We can reduce this model to obtain the standard Jakes' bathtub model. If we set the angle spread concentration to zero ($\kappa_n = 0$ for all n), the modified Bessel function $I_0(0) = 1$, and the exponential term evaluates to 1. The distribution reduces to:

$$p(\theta) = \frac{1}{2\pi}, \quad \theta \in (-\pi, \pi] \tag{4}$$

which is exactly Clarke's 2-D isotropic scattering model.

For every distinct path, there is 1 cluster. For a wideband signal, we can assume that there is 1 sub-cluster in each cluster. Keeping this in mind, $N = 1$. The temporal autocorrelation of the fading process hence is:

$$r_g(\tau) = \frac{I_0\left(\sqrt{\kappa^2 - 4\pi^2 f_m^2 \tau^2 + j4\pi\kappa f_m \tau \cos\phi}\right)}{I_0(\kappa)} \tag{5}$$

and its corresponding Doppler spectrum is:

$$S_g(f) = \frac{\exp(\kappa f \cos\phi / f_m)\cosh(\kappa\sqrt{1-(f/f_m)^2}\sin\phi)}{\pi\sqrt{f_m^2 - f^2} I_0(\kappa)}, \quad |f| \leq f_m \tag{6}$$

where f_m is the maximum Doppler frequency shift.

If we evaluate the autocorrelation function for a isotropic environment by setting $\kappa = 0$:

$$r_g(\tau) = I_0(j2\pi f_m \tau) \tag{7}$$

Using the Bessel function property $I_0(jx) = J_0(x)$, this yields the Jakes' autocorrelation:

$$r_g(\tau) = J_0(2\pi f_m \tau) \tag{8}$$

Similarly, setting $\kappa = 0$ in the spectrum equation yields $\cosh(0) = 1$ and $\exp(0) = 1$, perfectly reducing to the classic bathtub-shaped Jakes' Doppler spectrum:

$$S_g(f) = \frac{1}{\pi\sqrt{f_m^2 - f^2}}, \quad |f| \leq f_m \tag{9}$$

As discussed earlier, the modified SoS model generates the complex fading process $Z(t)$ as:

$$Z(t) = Z_c(t) + jZ_s(t) \tag{3a}$$

$$Z_c(t) = \sqrt{\frac{2}{M}} \sum_{n=1}^{M} \cos(w_d t \cos\alpha_n + \phi_n) \tag{3b}$$

$$Z_s(t) = \sqrt{\frac{2}{M}} \sum_{n=1}^{M} \cos(w_d t \sin\alpha_n + \varphi_n) \tag{3c}$$

with the discrete angles of arrival α_n typically defined as:

$$\alpha_n = \frac{2\pi n - \pi + \theta}{4M}, \quad n = 1, 2, \ldots, M \tag{4}$$

where ϕ_n, φ_n and θ are independent and uniformly distributed on $[-\pi, \pi)$ for all n.

In this standard modified SoS model, the physical scattering environment is assumed to be isotropic. This is reflected in Equation (4), which ensures that the angles of arrival, α_n, are uniformly spaced and distributed across the azimuth $[-\pi, \pi)$.

To implement the flexible von Mises Doppler model within this SoS framework, we must drop the uniform assumption for the angles of arrival. Instead of defining α_n via the uniform

mapping in Equation (4), the angles of arrival α_n are generated as random variables drawn directly from the mixture of von Mises distributions $p(\theta)$ defined in Equation (3).

By replacing the uniform angular distribution with the von Mises distribution, the SoS simulator shifts from modeling isotropic scattering to non-isotropic scattering.

After making this adjustment for the SoS model with $N = 1, \kappa = 5, \phi = 45$ cluster, we get the below plots

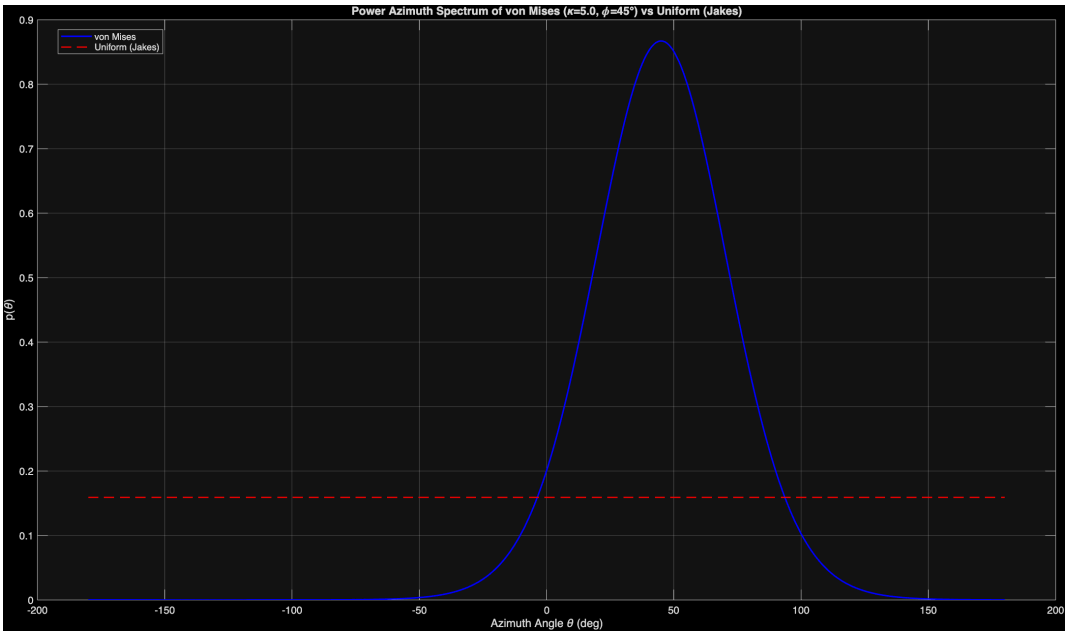

Figure 31: Custom PAS requiring a flexible Doppler model

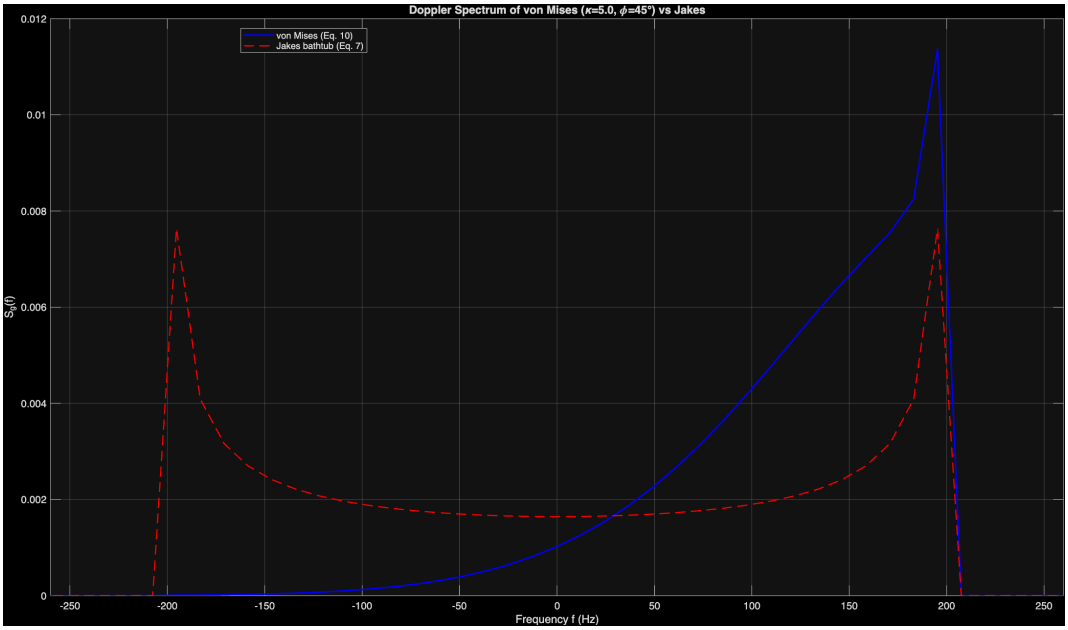

Figure 32: Flexible Doppler spread for given PAS

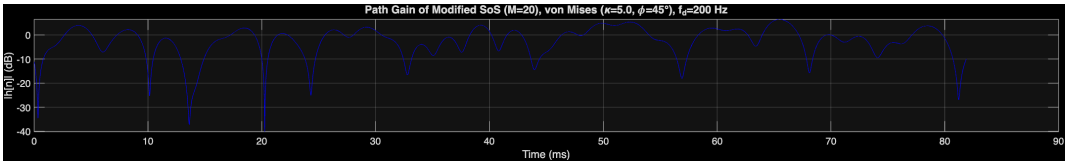

Figure 33: Path gain of time-domain channel

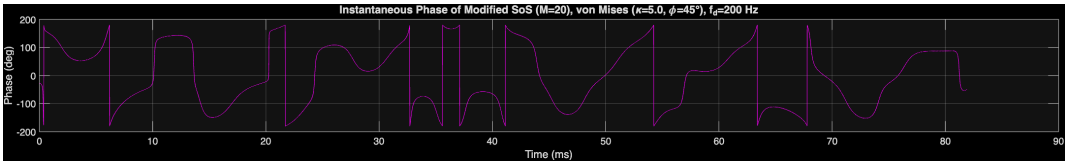

Figure 34: Phase of time-domain channel

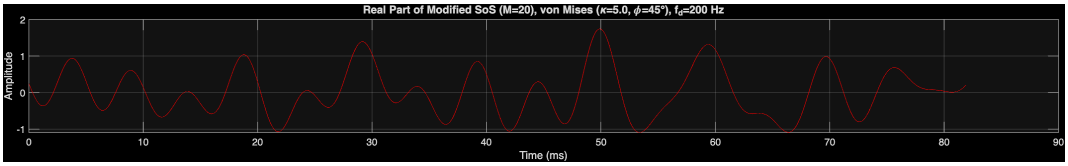

Figure 35: Real part of time-domain channel

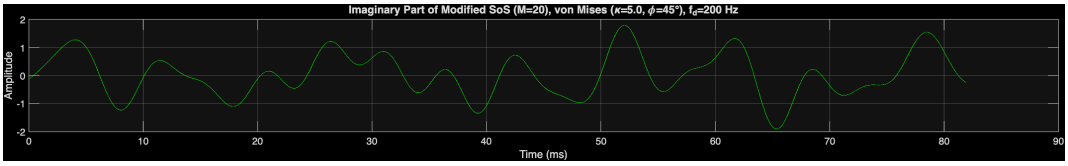

Figure 36: Imaginary part of time-domain channel

Question 2

1. **Question:** Consider a wide-band signal with pass-band bandwidth 2W=10MHz, which is transmitted over a multi-path model defined by the power delay profile (PDP) (given in the assignment handout).

   For the above PDP, take a 2048 point FFT of the instantaneous h[n] (by appropriate zero-padding) to get H[k]. Plot in dB scale the squared gain, i.e., $10\log_{10}(|H(k)|^2)$, to interpret the coherence band-width of the model and comment. Repeat over 3 different (independent) channel realisations, and plot on the same figure.

   Answer: In a multi-path model, each path has a certain path gain and a delay, specified by the given PDP. In addition, the exact small-scale fading characteristics of each path could be different. In the setting of the current question, we consider the following:

   - The sampling time $T_s = 1/2W$, which corresponds to sinc pulses being used.
   - The small-scale channel in a single path is a single complex Gaussian in the time domain. This is typically what is observed when there is no relative motion between the transmitter and the receiver, no carrier frequency offset, or other factors that contribute to a time-varying channel.

   The plot of the squared gain of the channel in the frequency domain is given below:

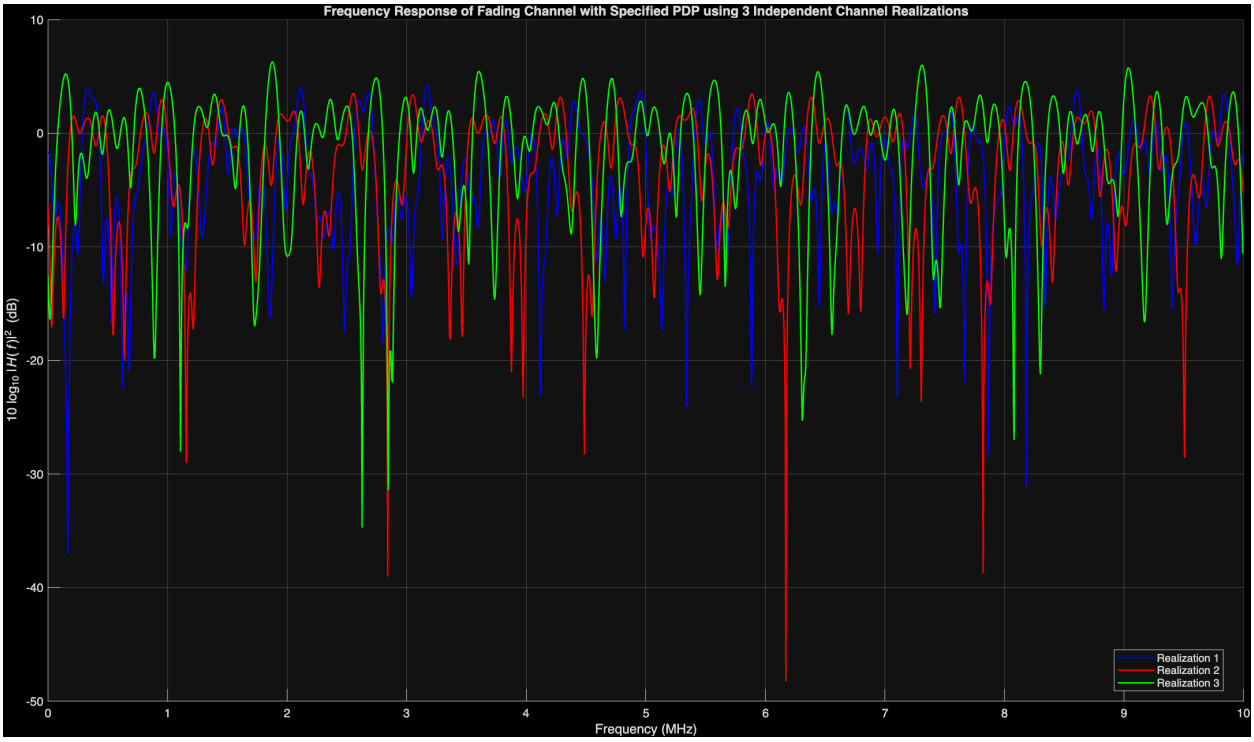

Figure 37: Magnitude of channel gain in frequency domain with given PDP

To calculate the coherence bandwidth (B_c), we first need to determine the RMS delay spread (σ_τ) from the given Power Delay Profile (PDP).

We convert the given path gains from dB to a linear scale ($P_i = 10^{\text{Gain}_{dB}/10}$) and then find the normalized linear gains ($\tilde{P}_i$) such that $\sum_i \tilde{P}_i = 1$.

$$\tilde{P}_i = \frac{P_i}{\sum_j P_j} \tag{10}$$

Applying this normalization gives us the following power delay profile:

Path Delay τ_i (μs)	**Gain (dB)**	**Linear Power P_i**	**Normalized Power $\tilde{P}_i$**
0	-2	0.6310	0.2220
1.8	0	1.0000	0.3518
3.5	-1	0.7943	0.2795
5.7	-6	0.2512	0.0884
8.1	-9	0.1259	0.0443
12.3	-14	0.0398	0.0140
		Sum:	**1.0000**

Table 1: Normalized Power Delay Profile

Because our powers are now normalized ($\sum_i \tilde{P}_i = 1$), the formulas for mean excess delay ($\bar{\tau}$) and the second central moment ($\overline{\tau^2}$) simplify to:

$$\bar{\tau} = \sum_i \tilde{P}_i \tau_i \tag{11}$$

$$\overline{\tau^2} = \sum_i \tilde{P}_i \tau_i^2 \tag{12}$$

Calculating the mean excess delay:

$$\bar{\tau} = 2.6464 \ \mu\text{s} \tag{13}$$

Calculating the second moment:

$$\overline{\tau^2} = 12.4604 \ (\mu\text{s})^2 \tag{14}$$

Now we calculate the RMS delay spread (σ_τ):

$$\begin{aligned} \sigma_\tau &= \sqrt{\overline{\tau^2} - (\bar{\tau})^2} \\ &\approx \sqrt{12.4604 - (2.6464)^2} \\ &\approx \sqrt{12.4604 - 7.0034} \\ &= \sqrt{5.4570} \approx 2.336 \ \mu\text{s} \end{aligned} \tag{15}$$

Finally, estimating the coherence bandwidth (B_c) using the correlation threshold of approximately 0.5:

$$B_c \approx \frac{1}{5\sigma_\tau} = \frac{1}{5 \times 2.336 \times 10^{-6}} \approx 85616 \text{ Hz} \approx 85.6 \text{ kHz} \tag{16}$$

Since the transmission pass-band bandwidth is $2W = 10$ which is vastly greater than the coherence bandwidth, the channel is highly frequency-selective. This is also shown in the plots below, which depict the variation of the channel in the frequency domain for 2 different resolutions.

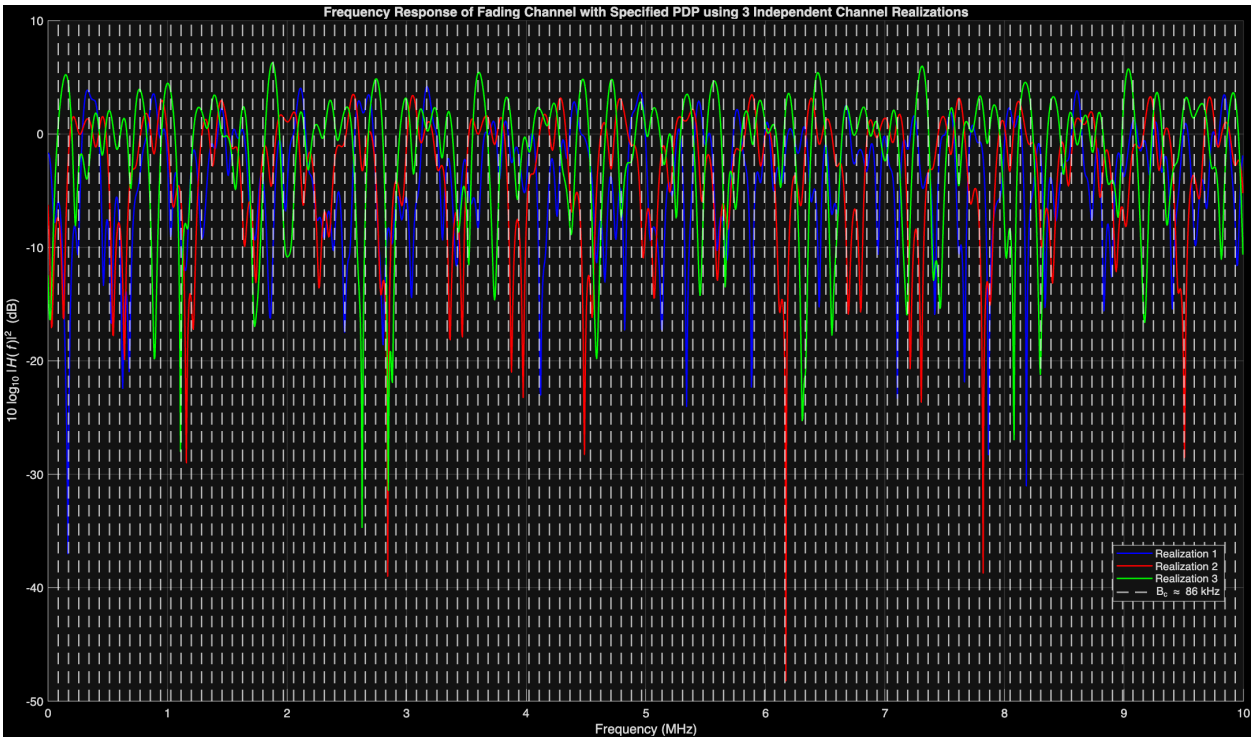

Figure 38: Caption

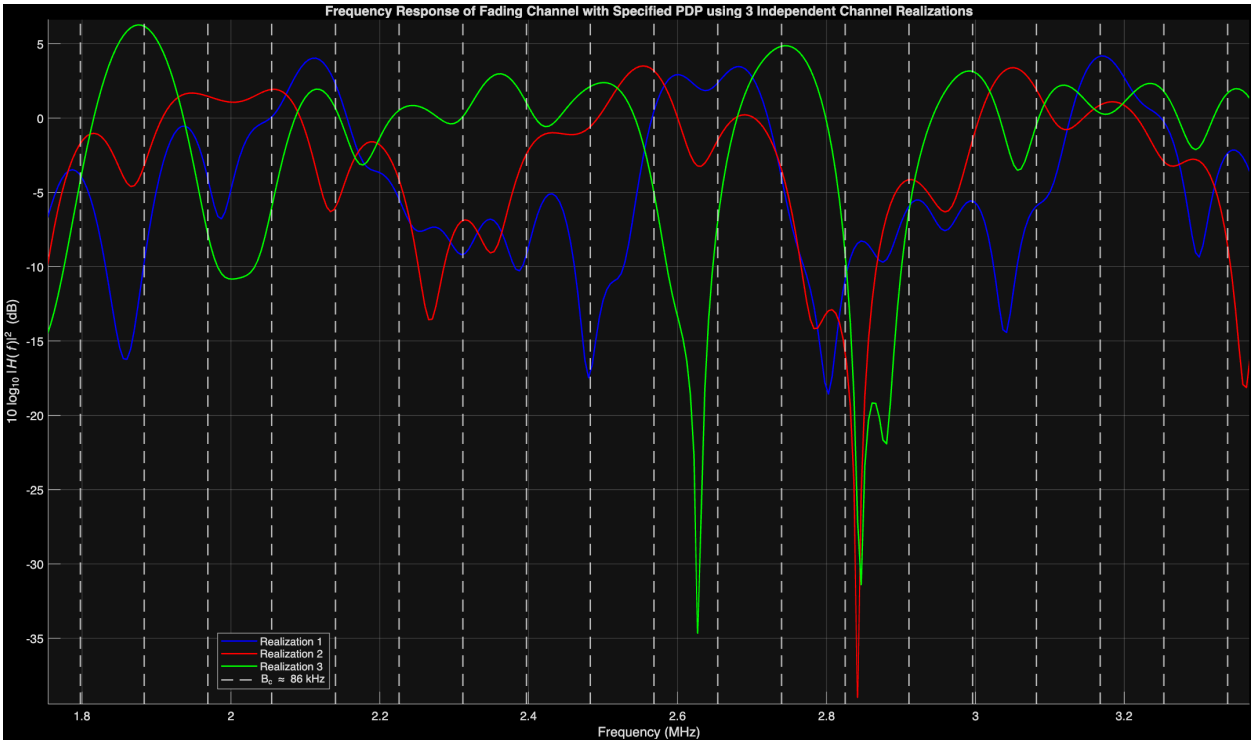

Figure 39: Caption

Question 3

1. **Question:** Now, for 90Hz maximum Doppler frequency on each path in the PDP as in Problem 2, generate frequency and time varying fading plots (use 3-D plotting feature in Matlab). Plot the time-varying frequency response, obtained every 5msec, over a duration of 30msecs (i.e., plot 7 consecutive snapshots, obtained at 0msec, 5msec, 10msec,. . . .., up to 30msec). Comment on your results.

Answer: We have seen, in the questions earlier, 2 different effects:

- Question 1 covered the time variation of the channel gain due to Doppler spread. We observed that the maximum Doppler frequency of operation determines if the channel is considered fast or slow fading.
- Question 2 covered the delay variation of the channel due to multipath. We observed that the Power Delay Profile determines if the channel is flat-fading or frequency-selective in the bandwidth of operation.

Now the most general case of the channel, which is covered partially in this question, is if there are multiple paths, each with independent and time-varying complex random processes in the time domain. Such a channel has both the aspects of Time variation/Doppler variation and Delay variation/Frequency variation, as can be seen in the time-frequency plot below.

As a result, such a channel can be represented in 4 different ways, and these are described by the Bello system functions [5]. The four Bello system functions are related by Fourier transforms between the time (t) / Doppler (ν) domains and the delay (τ) / frequency (f) domains:

1. **Time-varying impulse response:** $h(t, \tau)$
2. **Time-varying transfer function:**

$$H(t, f) = \int_{-\infty}^{\infty} h(t, \tau) e^{-j2\pi f \tau} d\tau \tag{17}$$

3. **Delay-Doppler spread function:**

$$S(\nu, \tau) = \int_{-\infty}^{\infty} h(t, \tau) e^{-j2\pi \nu t} dt \tag{18}$$

4. **Doppler-variant transfer function:**

$$B(\nu, f) = \int_{-\infty}^{\infty} H(t, f) e^{-j2\pi \nu t} dt = \int_{-\infty}^{\infty} S(\nu, \tau) e^{-j2\pi f \tau} d\tau \tag{19}$$

Each of the 4 visualizations are explored below, after first understanding the channel in the time and frequency domains. Each representation is useful for a certain use-case, and helps in understanding the variation of the channel with physical-layer parameters.

The 3D plot (with interpolation) of the snapshots across time, with each snapshot varying in frequency, is given below. The corresponding fading channel gain for each snapshot is also plotted below.

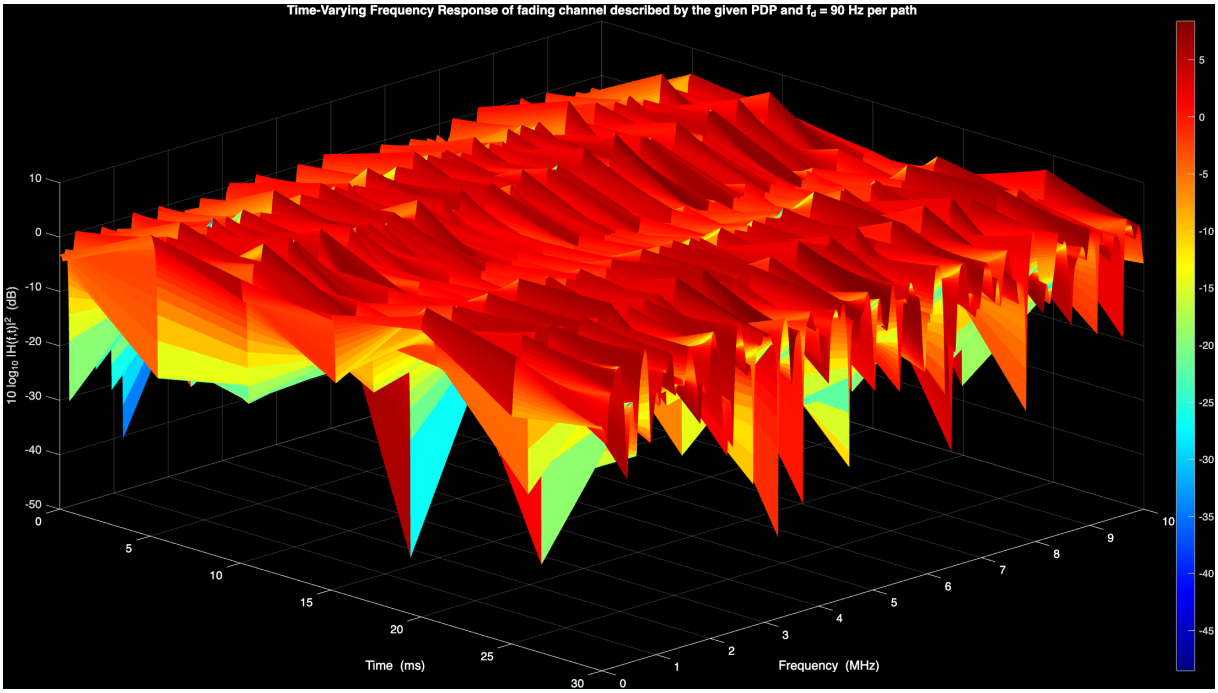

Figure 40: 3D Time-Frequency plot of multipath time-varying channel

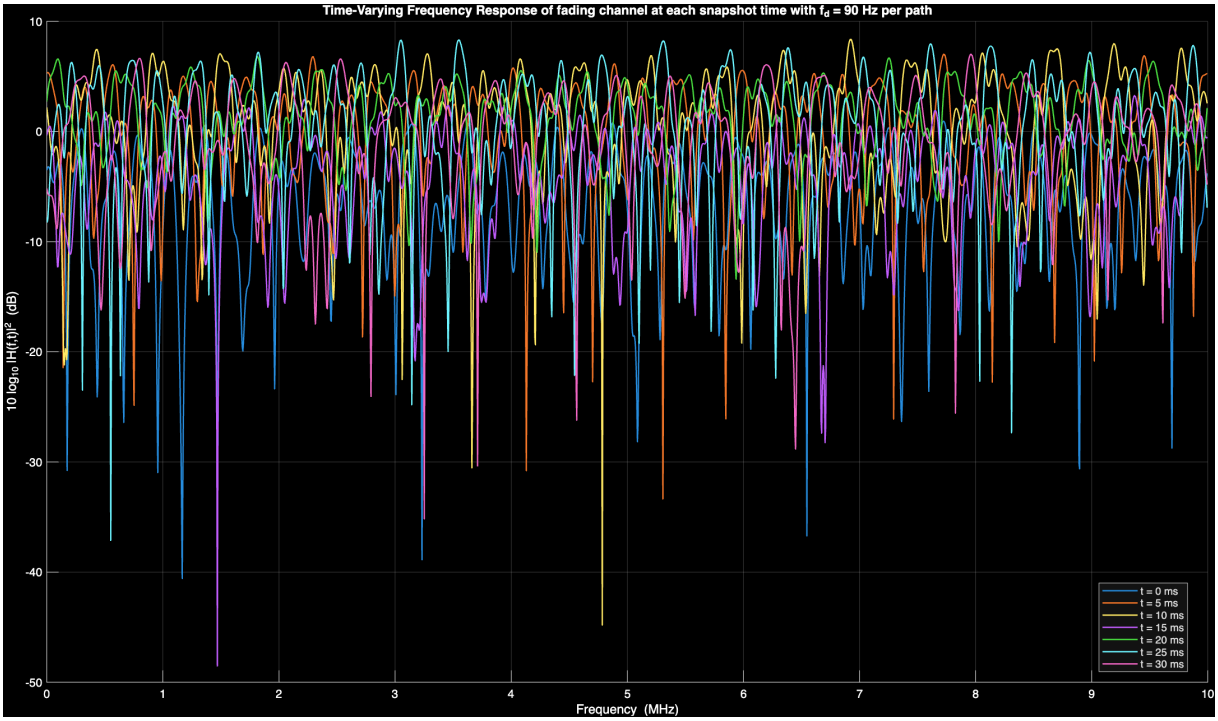

Figure 41: Channel gains of time-varying multipath channel

The 4 channel representations are provided below:

- **Time-varying impulse response ($h(t, \tau)$)**

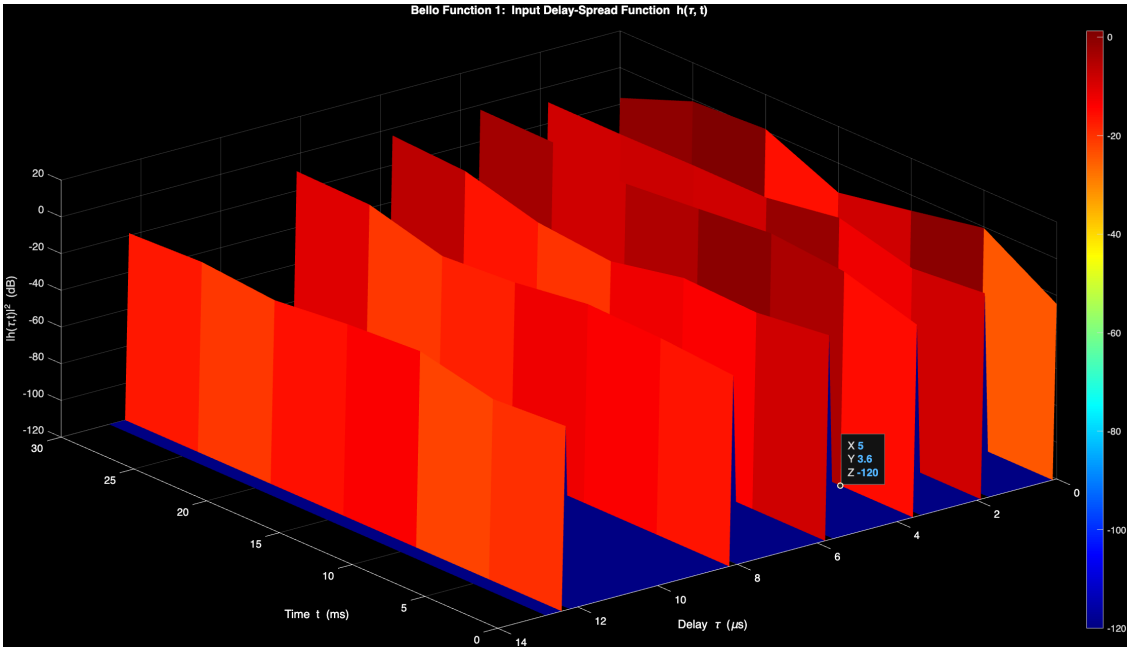

Figure 42: 3D plot of the time-varying impulse response $h(t, \tau)$ over the simulation duration.

- **Time-varying transfer function ($H(t, f)$)**

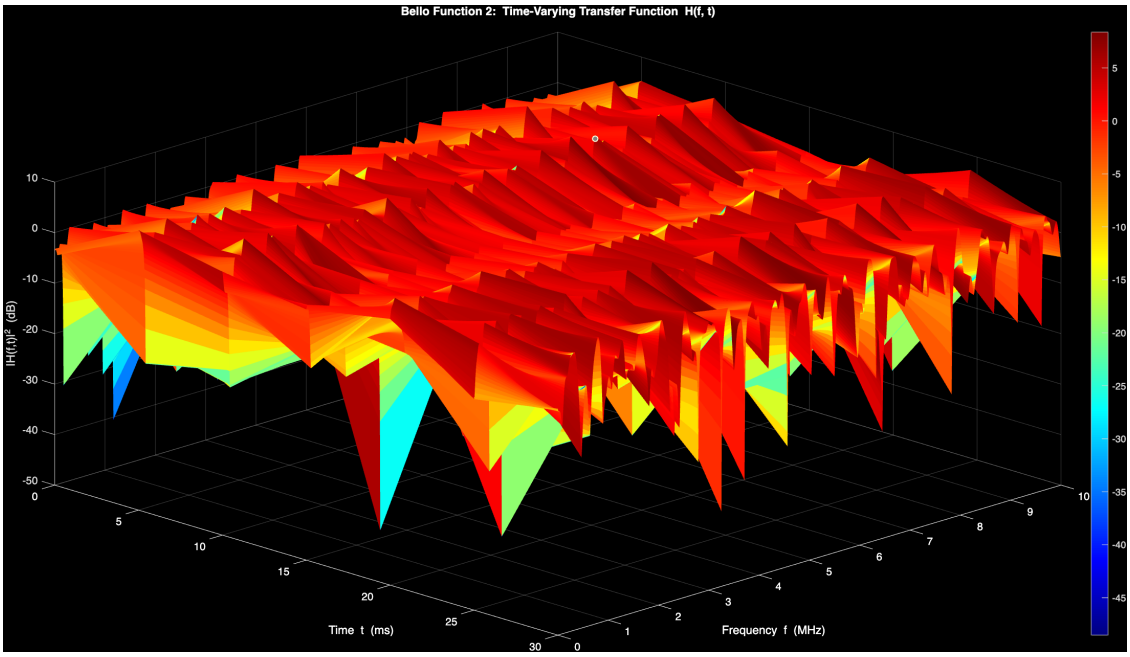

Figure 43: 3D plot of the time-varying frequency response $H(t, f)$ evaluated every 5 ms.

- **Delay-Doppler spread function ($S(\nu, \tau)$)**

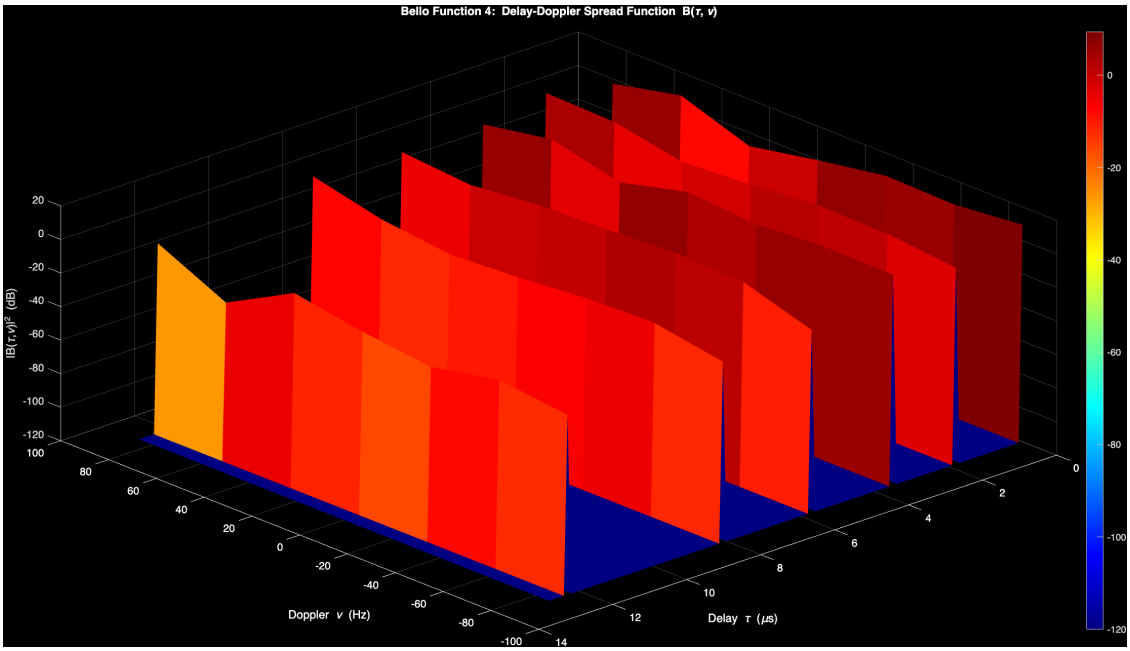

Figure 44: 3D plot of the Delay-Doppler spread function $S(\nu, \tau)$.

- **Doppler-variant transfer function ($B(\nu, f)$)**

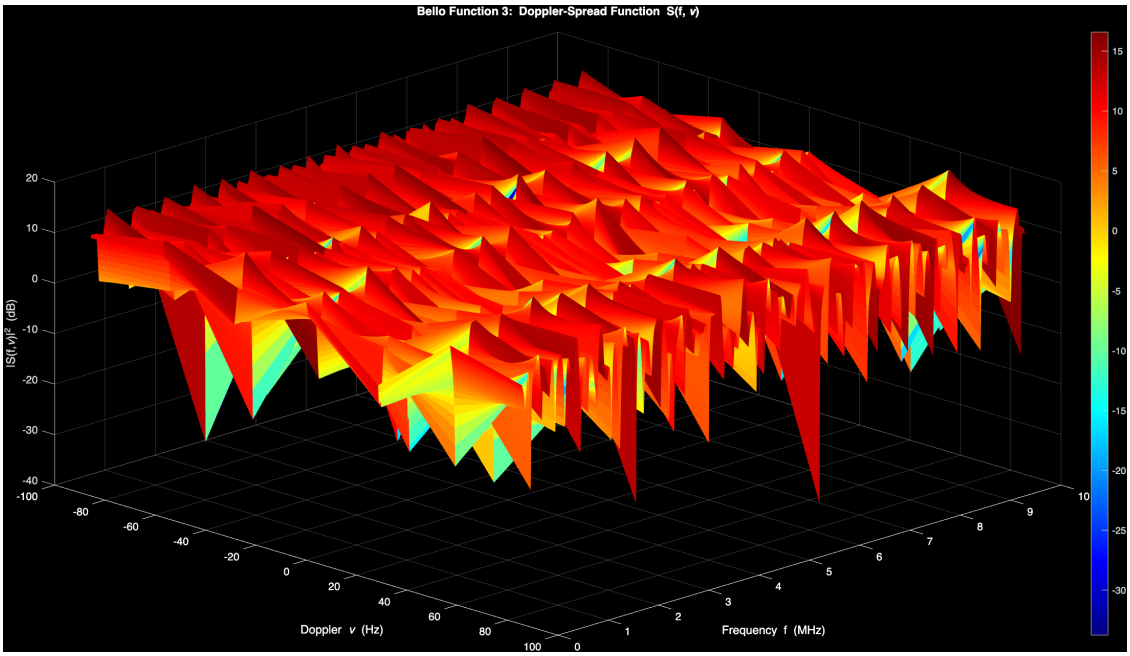

Figure 45: 3D plot of the Doppler-variant transfer function $B(\nu, f)$.

References

[1] J.B. Andersen, J.O. Nielsen, G.F. Pedersen, G. Bauch, and G. Dietl. Doppler spectrum from moving scatterers in a random environment. 8(6):3270–3277.

[2] R. H. Clarke. A statistical theory of mobile-radio reception. 47(6):957–1000.

[3] P. Dent, G.E. Bottomley, and T. Croft. Jakes fading model revisited. 29(13):1162–1163.

[4] J.I. Smith. A computer generated multipath fading simulation for mobile radio. 24(3):39–40.

[5] Emmanuel Tonyé. A comparative approach of doppler spectra for fading channel modelling by the filtered white gaussian noise method. 2015.

[6] Shuangquan Wang, Xiaodong Wang, and Mohammad Madihian. A flexible doppler behavior model.

[7] Y.R. Zheng and Chengshan Xiao. Improved models for the generation of multiple uncorrelated Rayleigh fading waveforms. 6(6):256–258.

A Codes

A.1 Q1_1a_levinson.m

```
%% Problem 1a(a): Levinson AR-based Noise Coloring Filter
clear; clc; close all;

v   = 30;           % velocity (m/s)
fc  = 2e9;          % carrier frequency (Hz)
c   = 3e8;          % speed of light (m/s)
W   = 50e3;         % one-sided bandwidth (2W = 100 kHz)
Ts  = 1 / (2*W);    % sampling period: 10 us
fd  = v * fc / c;   % max Doppler frequency: 200 Hz
N   = 8000;         % number of samples
P   = 100;          % AR model order (tune between 50-200)

lags   = 0:P;
R_comp = 0.5 * besselj(0, 2*pi * fd * Ts * lags);
R_comp(1) = R_comp(1) + 1e-6;
[a, sigma2] = levinson(R_comp, P);
fprintf('Prediction error variance sigma2 = %.6f\n', sigma2);

rng(42);
w_I = sqrt(sigma2) * randn(1, N);
w_Q = sqrt(sigma2) * randn(1, N);


h_I = filter(1, a, w_I);
h_Q = filter(1, a, w_Q);

h = h_I + 1j * h_Q;

t_ms = (0:N-1) * Ts * 1e3;

figure('Name', 'Levinson AR Fading | v=30 m/s');

subplot(4,1,1);
plot(t_ms, 20*log10(abs(h)), 'b');
xlabel('Time (ms)'); ylabel('Instantaneous power |{\it h}[n]|^2 (dB)', 'Interpreter', 'tex');
title_str = sprintf('Path Gain Envelope for an AR(%d) process with {\\itv} = %d m/s or {\\itf}
title(title_str, 'Interpreter', 'tex');
grid on;

subplot(4,1,2);
plot(t_ms, angle(h) * 180/pi, 'm');
xlabel('Time (ms)');
ylabel('Phase (deg)');
title_str_phase1 = sprintf('Instantaneous Phase of the AR(%d) process with {\\itv} = %d m/s or
```

```
title(title_str_phase1, 'Interpreter', 'tex');
grid on;


subplot(4,1,3);
plot(t_ms, h_I, 'r');
xlabel('Time (ms)'); ylabel('Amplitude');
title_str2 = sprintf('Real part (I component) of the AR(%d) process with {\\itv} = %d m/s or {'
title(title_str2, 'Interpreter', 'tex');
grid on;

subplot(4,1,4);
plot(t_ms, h_Q, 'g');
xlabel('Time (ms)'); ylabel('Amplitude');
title_str3 = sprintf('Imaginary part (Q component) of the AR(%d) process with {\\itv} = %d m/s
title(title_str3, 'Interpreter', 'tex');
grid on;

Ns   = length(h);
Nlag     = min(40000, Ns-1);
[acf, lags] = xcorr(h,   Nlag, 'normalized');
[xc,  ~   ] = xcorr(h_I, h_Q, Nlag, 'normalized');

tau_ms   = lags * Ts * 1e3;
R_theory = besselj(0, 2*pi * fd * abs(lags) * Ts);

figure('Name', 'ACF and Cross-Correlation: Levinson Method');

subplot(1,2,1);
plot(tau_ms, real(acf), 'b', 'LineWidth', 1.2); hold on;
plot(tau_ms, R_theory,  'r--', 'LineWidth', 1.2);
xlabel('Time Lag, {\it\tau} (ms)', 'Interpreter', 'tex');
ylabel('{\itR_h}[{\it\tau}]', 'Interpreter', 'tex');
title_str1 = sprintf('Autocorrelation (ACF) of the AR(%d) process with {\\itv} = %d m/s or {\\i
title(title_str1, 'Interpreter', 'tex');
legend('Simulated (Real Part)', 'Theoretical: {\itJ}_0(2\pi{\itf}_d{\it\tau})', 'Interpreter',
grid on;

xc_real  = real(xc);
rms_xc   = sqrt(mean(xc_real.^2));

subplot(1,2,2);
plot(tau_ms, xc_real, 'b', 'LineWidth', 1.2); hold on;
yline(0, 'r--', 'LineWidth', 1.2);
xlabel('Time Lag, {\it\tau} (ms)', 'Interpreter', 'tex');
ylabel('{\itR}_{IQ}[{\it\tau}]', 'Interpreter', 'tex');

title_str2 = sprintf('Cross-Correlation (I vs Q) of the AR(%d) process with {\\itv} = %d m/s o
```

```
title(title_str2, 'Interpreter', 'tex');
legend('Simulated', 'Ideal (0)', 'Interpreter', 'tex');
grid on;

% Add RMS Text Box
text(0.02, 0.95, sprintf('RMS = %.4f', rms_xc), ...
    'Units', 'normalized', 'VerticalAlignment', 'top', ...
    'FontSize', 12, 'Color', 'w', 'BackgroundColor', 'k', 'EdgeColor', 'k');


%% ---- Second Independent Path ----
% Remark 2 in Zheng & Xiao: using distinct (theta_k, phi_nk, varphi_nk)
% creates uncorrelated faders with identical statistical properties

rng(99);
w_I2 = sqrt(sigma2) * randn(1, N);
w_Q2 = sqrt(sigma2) * randn(1, N);
h_I2 = filter(1, a, w_I2);
h_Q2 = filter(1, a, w_Q2);

h2 = h_I2 + 1j * h_Q2;
[xc_paths, lags_p] = xcorr(h, h2, Nlag, 'normalized');
xc_paths_real = real(xc_paths);
rms_paths     = sqrt(mean(xc_paths_real.^2));
tau_ms_p      = lags_p * Ts * 1e3;

%% ---- Inter-Path Correlation Plot ----
figure('Name', 'Inter-Path Correlation: Path 1 vs Path 2');

subplot(1,2,1);
plot(t_ms, 20*log10(abs(h)),  'b', 'LineWidth', 1);  hold on;
plot(t_ms, 20*log10(abs(h2)), 'r', 'LineWidth', 1);
xlabel('Time (ms)');
ylabel('|h[n]| (dB)');
title_str_p = sprintf('Two Independent Path Gains of the AR(%d) process with {\\itv} = %d m/s
title(title_str_p, 'Interpreter', 'tex');
legend('Path 1', 'Path 2');
grid on;

subplot(1,2,2);
plot(tau_ms_p, xc_paths_real, 'w', 'LineWidth', 1.2);  hold on;
yline(0, 'r--', 'LineWidth', 1.2);
xlabel('Time Lag, {\it\tau} (ms)', 'Interpreter', 'tex');
ylabel('R_{h_1 h_2}[\tau]', 'Interpreter', 'tex');
title_str_xc = sprintf('Cross-Correlation of Path 1 vs Path 2, each of the AR(%d) process with
title(title_str_xc, 'Interpreter', 'tex');
legend('Simulated', 'Ideal (0)', 'Interpreter', 'tex');
grid on;
```

```
text(0.02, 0.95, sprintf('RMS = %.4f', rms_paths), ...
    'Units', 'normalized', 'VerticalAlignment', 'top', ...
    'FontSize', 12, 'Color', 'w', 'BackgroundColor', 'k', 'EdgeColor', 'k');
```

A.2 Q1_1a_smith.m

```
%% Problem 1a(b): Smith's Model (FFT-based) Fading Simulator
clear; clc; close all;

%% Parameters
v  = 30;            % velocity (m/s)
fc = 2e9;           % carrier frequency (Hz)
c  = 3e8;           % speed of light (m/s)
W  = 50e3;          % one-sided bandwidth (Hz)
Ts = 1/(2W);       % sampling period (s)
fd = vfc/c;        % max Doppler (Hz)
N  = 8192;          % FFT size
fs = 1/Ts;          % sampling frequency
df = fs/N;          % frequency resolution


%% Jakes' PSD
f = (-N/2 : N/2-1) * df;
S = zeros(1, N);
idx = abs(f) < fd;
S(idx) = 1 ./ (pi * fd * sqrt(1 - (f(idx)/fd).^2));

S(~isfinite(S)) = max(S(isfinite(S))); % Handle singularity at ±fd
S = S / (sum(S) * df);                 % Normalize to unit power

%% Complex Gaussian & Spectral Shaping
rng(42);
X = sqrt(N) * (randn(1,N) + 1jrandn(1,N)) / sqrt(2);
Y = X . sqrt(S * df);

h = N * ifft(ifftshift(Y));
h = h / sqrt(mean(abs(h).^2)); % Normalize to unit average power
h_I = real(h);
h_Q = imag(h);

%% Plot: Time-domain
t_ms = (0:N-1) * Ts * 1e3;

figure('Name', "Smith's FFT Fading");
subplot(4,1,1); plot(t_ms, 20*log10(abs(h)), 'b'); grid on;
xlabel('Time (ms)'); ylabel('|{\it h}[n]|^2 (dB)', 'Interpreter', 'tex');
title(sprintf('Path Gain Envelope (N=%d, {\itv} = %d m/s)', N, v), 'Interpreter', 'tex');
```

```
subplot(4,1,2); plot(t_ms, angle(h) * 180/pi, 'm'); grid on;
xlabel('Time (ms)'); ylabel('Phase (deg)');
title(sprintf('Instantaneous Phase (N=%d, {\itv} = %d m/s)', N, v), 'Interpreter', 'tex');

subplot(4,1,3); plot(t_ms, h_I, 'r'); grid on;
xlabel('Time (ms)'); ylabel('Amplitude');
title('Real part (I component)', 'Interpreter', 'tex');

subplot(4,1,4); plot(t_ms, h_Q, 'g'); grid on;
xlabel('Time (ms)'); ylabel('Amplitude');
title('Imaginary part (Q component)', 'Interpreter', 'tex');

%% ACF and Cross-correlation
Nlag = min(20000, length(h)-1);
[acf, lags_out] = xcorr(h, Nlag, 'normalized');
[xc, ~] = xcorr(h_I, h_Q, Nlag, 'normalized');

tau_ms = lags_out * Ts * 1e3;
R_theory = besselj(0, 2*pi * fd * abs(lags_out) * Ts);

figure('Name', 'ACF and Cross-Correlation: Smith''s Method');
subplot(1,2,1);
plot(tau_ms, real(acf), 'b', tau_ms, R_theory, 'r--', 'LineWidth', 1.2); grid on;
xlabel('Time Lag, {\it\tau} (ms)', 'Interpreter', 'tex'); ylabel('{\itR_h}[{\it\tau}]', 'Interp
title('Autocorrelation (ACF)', 'Interpreter', 'tex');
legend('Simulated (Real Part)', 'Theoretical');

xc_real = real(xc);
rms_xc = sqrt(mean(xc_real.^2));

subplot(1,2,2);
plot(tau_ms, xc_real, 'b', 'LineWidth', 1.2); hold on;
yline(0, 'r--', 'LineWidth', 1.2); grid on;
xlabel('Time Lag, {\it\tau} (ms)', 'Interpreter', 'tex'); ylabel('{\itR}_{IQ}[{\it\tau}]', 'Int
title('Cross-Correlation (I vs Q)', 'Interpreter', 'tex');
legend('Simulated', 'Ideal (0)');
text(0.02, 0.95, sprintf('RMS = %.4f', rms_xc), 'Units', 'normalized', 'VerticalAlignment', 'to

%% Second Independent Path
rng(99);
X2 = (randn(1,N) + 1jrandn(1,N)) / sqrt(2);
Y2 = X2 . sqrt(S * df);
h2 = N * ifft(ifftshift(Y2));
h2 = h2 / sqrt(mean(abs(h2).^2));

[xc_paths, lags_p] = xcorr(h, h2, Nlag, 'normalized');
xc_paths_real = real(xc_paths);
rms_paths = sqrt(mean(xc_paths_real.^2));
```

```
figure('Name', 'Inter-Path Correlation: Smith''s Method');
subplot(1,2,1);
plot(t_ms, 20log10(abs(h)), 'b', t_ms, 20log10(abs(h2)), 'r', 'LineWidth', 1); grid on;
xlabel('Time (ms)'); ylabel('|h[n]| (dB)');
title('Independent Path Gains'); legend('Path 1', 'Path 2');

subplot(1,2,2);
plot(lags_p * Ts * 1e3, xc_paths_real, 'k', 'LineWidth', 1.2); hold on;
yline(0, 'r--', 'LineWidth', 1.2); grid on;
xlabel('Time Lag, {\it\tau} (ms)', 'Interpreter', 'tex'); ylabel('R_{h_1 h_2}[\tau]', 'Interpre
title('Cross-Correlation (Path 1 vs 2)', 'Interpreter', 'tex');
text(0.02, 0.95, sprintf('RMS = %.4f', rms_paths), 'Units', 'normalized', 'VerticalAlignment',
```

A.3 Q1_1a_sos.m

```
%% Problem 1a(c): Modified Sum-of-Sinusoids | Zheng & Xiao (2002)
clear; clc; close all;

v  = 30;            % velocity (m/s)
fc = 2e9;           % carrier frequency (Hz)
c  = 3e8;           % speed of light (m/s)
W  = 50e3;          % one-sided bandwidth
Ts = 1/(2*W);       % sampling period: 10 us
fd = v*fc/c;        % max Doppler: 200 Hz
wd = 2*pi*fd;       % max angular Doppler (rad/s)
N  = 8000;          % number of samples
M  = 20;            % sinusoids per component

t = (0:N-1) * Ts;       % (1 x N) in seconds

rng(42);
theta    = unifrnd(-pi, pi);
phi_n    = unifrnd(-pi, pi, 1, M);
varphi_n = unifrnd(-pi, pi, 1, M);

n       = (1:M);
alpha_n = (2*pi*n - pi + theta) / (4*M);

t_col = t(:);                                                % (N x 1)

Z_c = sqrt(2/M) * sum(cos(wd * t_col * cos(alpha_n) + phi_n),    2)';
Z_s = sqrt(2/M) * sum(cos(wd * t_col * sin(alpha_n) + varphi_n), 2)';

h = Z_c + 1j * Z_s;

%% ---- Plot ----
t_ms = t * 1e3;
```

```
figure('Name', 'SOS Fading | v=30 m/s');

subplot(4,1,1);
plot(t_ms, 20*log10(abs(h)), 'b');
xlabel('Time (ms)'); ylabel('Instantaneous power |{\it h}[n]|^2 (dB)', 'Interpreter', 'tex');
title_str = sprintf('Path Gain Envelope of the channel with M=%d sinusoids for {\\itv} = %d m/s
title(title_str, 'Interpreter', 'tex');
grid on;

subplot(4,1,2);
plot(t_ms, angle(h) * 180/pi, 'm');
xlabel('Time (ms)');
ylabel('Phase (deg)');
title_str_phase3 = sprintf('Instantaneous Phase of the channel with M=%d sinusoids for {\\itv}
title(title_str_phase3, 'Interpreter', 'tex');
grid on;

subplot(4,1,3);
plot(t_ms, Z_c, 'r');
xlabel('Time (ms)'); ylabel('Amplitude');
title_str2 = sprintf('Real part (I component) of the channel with M=%d sinusoids for {\\itv} =
title(title_str2, 'Interpreter', 'tex');
grid on;

subplot(4,1,4);
plot(t_ms, Z_s, 'g');
xlabel('Time (ms)'); ylabel('Amplitude');
title_str3 = sprintf('Imaginary part (Q component) of the channel with M=%d sinusoids for {\\it
title(title_str3, 'Interpreter', 'tex');
grid on;

%% ACF and Cross-correlation
h_I = Z_c;
h_Q = Z_s;
Ns   = length(h);
Nlag     = min(40000, Ns-1);
[acf, lags] = xcorr(h,   Nlag, 'normalized');
[xc,  ~   ] = xcorr(h_I, h_Q, Nlag, 'normalized');

tau_ms   = lags * Ts * 1e3;
R_theory = besselj(0, 2*pi * fd * abs(lags) * Ts);

figure('Name', 'ACF and Cross-Correlation of SOS');

subplot(1,2,1);
plot(tau_ms, real(acf), 'b', 'LineWidth', 1.2); hold on;
plot(tau_ms, R_theory,  'r--', 'LineWidth', 1.2);
```

```
xlabel('Time Lag, {\it\tau} (ms)', 'Interpreter', 'tex');
ylabel('{\itR_h}[{\it\tau}]', 'Interpreter', 'tex');
title_str1 = sprintf('Autocorrelation (ACF) of the M=%d process with {\\itv} = %d m/s or {\\it
title(title_str1, 'Interpreter', 'tex');
legend('Simulated (Real Part)', 'Theoretical: {\itJ}_0(2\pi{\itf}_d{\it\tau})', 'Interpreter',
grid on;

xc_real  = real(xc);
rms_xc   = sqrt(mean(xc_real.^2));

subplot(1,2,2);
plot(tau_ms, xc_real, 'b', 'LineWidth', 1.2); hold on;
yline(0, 'r--', 'LineWidth', 1.2);
xlabel('Time Lag, {\it\tau} (ms)', 'Interpreter', 'tex');
ylabel('{\itR}_{IQ}[{\it\tau}]', 'Interpreter', 'tex');

title_str2 = sprintf('Cross-Correlation (I vs Q) of the M=%d process with {\\itv} = %d m/s or
title(title_str2, 'Interpreter', 'tex');
legend('Simulated', 'Ideal (0)', 'Interpreter', 'tex');
grid on;

% RMS Text Box
text(0.02, 0.95, sprintf('RMS = %.4f', rms_xc), ...
    'Units', 'normalized', 'VerticalAlignment', 'top', ...
    'FontSize', 12, 'Color', 'w', 'BackgroundColor', 'k', 'EdgeColor', 'k');

%% ---- Second Independent Path ----

rng(99);   % different seed
theta2    = unifrnd(-pi, pi);
phi_n2    = unifrnd(-pi, pi, 1, M);
varphi_n2 = unifrnd(-pi, pi, 1, M);

alpha_n2 = (2*pi*n - pi + theta2) / (4*M);

Z_c2 = sqrt(2/M) * sum(cos(wd * t_col * cos(alpha_n2) + phi_n2),    2)';
Z_s2 = sqrt(2/M) * sum(cos(wd * t_col * sin(alpha_n2) + varphi_n2), 2)';

h2 = Z_c2 + 1j * Z_s2;

[xc_paths, lags_p] = xcorr(h, h2, Nlag, 'normalized');
xc_paths_real = real(xc_paths);
rms_paths     = sqrt(mean(xc_paths_real.^2));
tau_ms_p      = lags_p * Ts * 1e3;

fprintf('\n--- Inter-Path Correlation (Path 1 vs Path 2) ---\n');
fprintf('  RMS of cross-correlation: %.6f  (ideal = 0)\n', rms_paths);
```

```
figure('Name', 'Inter-Path Correlation: Path 1 vs Path 2');

subplot(1,2,1);
plot(t_ms, 20*log10(abs(h)),  'b', 'LineWidth', 1);  hold on;
plot(t_ms, 20*log10(abs(h2)), 'r', 'LineWidth', 1);
xlabel('Time (ms)');
ylabel('|h[n]| (dB)');
title_str_p = sprintf('Two Independent Path Gains with M=%d, {\\itv}=%d m/s, {\\itf}_d=%.0f Hz
title(title_str_p, 'Interpreter', 'tex');
legend('Path 1', 'Path 2');
grid on;

subplot(1,2,2);
plot(tau_ms_p, xc_paths_real, 'w', 'LineWidth', 1.2);  hold on;
yline(0, 'r--', 'LineWidth', 1.2);
xlabel('Time Lag, {\it\tau} (ms)', 'Interpreter', 'tex');
ylabel('R_{h_1 h_2}[\tau]', 'Interpreter', 'tex');
title_str_xc = sprintf('Cross-Correlation of Path 1 vs Path 2, each with M=%d, {\\itv}=%d m/s v
title(title_str_xc, 'Interpreter', 'tex');
legend('Simulated', 'Ideal (0)', 'Interpreter', 'tex');
grid on;
text(0.02, 0.95, sprintf('RMS = %.4f', rms_paths), ...
    'Units', 'normalized', 'VerticalAlignment', 'top', ...
    'FontSize', 12, 'Color', 'w', 'BackgroundColor', 'k', 'EdgeColor', 'k');
```

A.4 Q1_Bonus.m

```
%% Flexible Doppler Behavior Model | von Mises PAS + Modified SoS
clear; clc; close all;


v   = 30;           % mobile speed (m/s)
fc  = 2e9;          % carrier frequency (Hz)
c   = 3e8;          % speed of light (m/s)
W   = 50e3;         % one-sided bandwidth (Hz)
Ts  = 1/(2*W);      % sampling period (s)
fd  = v*fc/c;       % max Doppler frequency (Hz)
wd  = 2*pi*fd;      % angular Doppler (rad/s)
N   = 8192;         % number of time samples
fs  = 1/Ts;         % sampling frequency (Hz)

%% ---- von Mises PAS Parameters (single cluster N=1) ----
kappa = 5;
phi   = pi/4;

t    = (0:N-1) * Ts;
t_ms = t * 1e3;
Nlag = min(4000, N-1);
```

```
theta_ax = linspace(-pi, pi, 1000);
pas_vm   = exp(kappa * cos(theta_ax - phi)) / (2*pi*besseli(0, kappa));
pas_unif = ones(size(theta_ax)) / (2*pi);

figure('Name', 'Fig 1: PAS Comparison');
plot(theta_ax*180/pi, pas_vm,   'b',  'LineWidth', 1.5); hold on;
plot(theta_ax*180/pi, pas_unif, 'r--','LineWidth', 1.2);
xlabel('Azimuth Angle \theta (deg)');
ylabel('p(\theta)');
title(sprintf('Power Azimuth Spectrum of von Mises (\\kappa=%.1f, \\phi=%.0f°) vs Uniform (Jake
legend('von Mises', 'Uniform (Jakes)', 'Location', 'best');
grid on;

df    = fs / N;
f_ax  = (-N/2 : N/2-1) * df;
idx   = abs(f_ax) < fd;
f_in  = f_ax(idx);
ratio = f_in / fd;
sq1mr = sqrt(1 - ratio.^2);

% von Mises Doppler spectrum
S_vm = zeros(1, N);
S_vm(idx) = exp(kappa * ratio * cos(phi)) .* ...
            cosh(kappa * sq1mr * sin(phi)) ./ ...
            (pi * fd * sq1mr * besseli(0, kappa));
S_vm(~isfinite(S_vm)) = max(S_vm(isfinite(S_vm)));
S_vm_norm = S_vm / (sum(S_vm)*df);          % normalize to unit power

% Jakes' Doppler spectrum
S_jk = zeros(1, N);
S_jk(idx) = 1 ./ (pi * fd * sqrt(1 - (f_in/fd).^2));
S_jk(~isfinite(S_jk)) = max(S_jk(isfinite(S_jk)));
S_jk_norm = S_jk / (sum(S_jk)*df);

figure('Name', 'Fig 2: Doppler Spectrum');
plot(f_ax, S_vm_norm, 'b',  'LineWidth', 1.5); hold on;
plot(f_ax, S_jk_norm, 'r--','LineWidth', 1.2);
xlim([-1.3*fd, 1.3*fd]);
xlabel('Frequency f (Hz)');  ylabel('S_g(f)');
title(sprintf('Doppler Spectrum of von Mises (\\kappa=%.1f, \\phi=%.0f°) vs Jakes', kappa, phi
legend('von Mises (Eq. 10)', 'Jakes bathtub (Eq. 7)', 'Location', 'best');
grid on;

lags_vec   = -Nlag:Nlag;
signed_tau = lags_vec * Ts;
tau_abs    = abs(lags_vec) * Ts;
tau_ms     = lags_vec * Ts * 1e3;
```

```
% von Mises ACF
% r_g(tau) = I0(sqrt(kappa^2 - 4*pi^2*fd^2*tau^2 + j*4*pi*kappa*fd*tau*cos(phi))) / I0(kappa)
arg_vm  = sqrt(kappa^2 - 4*pi^2*fd^2*signed_tau.^2 + ...
               1j*4*pi*kappa*fd*signed_tau*cos(phi));
R_vm_th = besseli(0, arg_vm) / besseli(0, kappa);

% Jakes ACF\
R_jk_th = besselj(0, 2*pi*fd*tau_abs);

M_sos = 20; % Number of sinusoids

% Draw arrival angles from von Mises(phi, kappa) instead of uniform
rng(42);
alpha_n = vmrnd(phi, kappa, 1, M_sos);

% Generate TWO independent sets of random phases ~ Uniform[-pi, pi)
phi_n    = unifrnd(-pi, pi, 1, M_sos);  % For In-phase
varphi_n = unifrnd(-pi, pi, 1, M_sos);  % For Quadrature

% 3. Calculate Z_c (In-phase) and Z_s (Quadrature) using SoS
t_col = t(:); % column vector (N x 1) for matrix expansion
Z_c = sqrt(2/M_sos) * sum(cos(wd * t_col * cos(alpha_n) + phi_n), 2).'; % (1 x N)
Z_s = sqrt(2/M_sos) * sum(cos(wd * t_col * sin(alpha_n) + varphi_n), 2).'; % (1 x N)

h_sos = Z_c + 1j * Z_s;
h_sos = h_sos / sqrt(mean(abs(h_sos).^2));
Z_c = real(h_sos);
Z_s = imag(h_sos);

figure('Name', 'Fig 3: SoS | von Mises Fading');
subplot(4,1,1);
plot(t_ms, 20*log10(abs(h_sos)), 'b');
xlabel('Time (ms)'); ylabel('|h[n]| (dB)');
title(sprintf('Path Gain of Modified SoS (M=%d), von Mises (\\kappa=%.1f, \\phi=%.0f°), f_d=%.0
grid on;
subplot(4,1,2);
plot(t_ms, angle(h_sos)*180/pi, 'm');
xlabel('Time (ms)'); ylabel('Phase (deg)');
title(sprintf('Instantaneous Phase of Modified SoS (M=%d), von Mises (\\kappa=%.1f, \\phi=%.0f°
grid on;
subplot(4,1,3);
plot(t_ms, Z_c, 'r');
xlabel('Time (ms)'); ylabel('Amplitude');
title(sprintf('Real Part of Modified SoS (M=%d), von Mises (\\kappa=%.1f, \\phi=%.0f°), f_d=%.0
grid on;
subplot(4,1,4);
plot(t_ms, Z_s, 'g');
```

```
xlabel('Time (ms)'); ylabel('Amplitude');
title(sprintf('Imaginary Part of Modified SoS (M=%d), von Mises (\\kappa=%.1f, \\phi=%.0f°), f
grid on;

%
%% LOCAL FUNCTION: von Mises random variate generator
function theta = vmrnd(mu, kap, varargin)
%VMRND  Generate von Mises distributed random variates.
%   theta = vmrnd(mu, kappa, m, n)
%   mu    : mean direction (rad)
%   kappa : concentration (>= 0);  kappa=0 -> uniform on [-pi,pi)
%
    if kap < 1e-6
        theta = -pi + 2*pi*rand(varargin{:});
        return;
    end
    sz   = [varargin{:}];
    nTot = prod(sz);

    tau_bf = 1 + sqrt(1 + 4*kap^2);
    rho    = (tau_bf - sqrt(2*tau_bf)) / (2*kap);
    r_bf   = (1 + rho^2) / (2*rho);

    theta = zeros(1, nTot);
    cnt   = 0;
    while cnt < nTot
        u1 = rand;
        z  = cos(pi * u1);
        f  = (1 + r_bf*z) / (r_bf + z);
        cc = kap * (r_bf - f);
        u2 = rand;
        if cc*(2 - cc) > u2 || log(cc/u2) + 1 >= cc
            u3  = rand;
            cnt = cnt + 1;
            theta(cnt) = sign(u3 - 0.5) * acos(f);
        end
    end
    theta = mu + reshape(theta, sz);
    theta = mod(theta + pi, 2*pi) - pi;      % wrap to [-pi, pi]
end
```

A.5 Q2_PDP.m

```
%% Problem 2: Frequency-Selective Fading | Power Delay Profile
clear; clc; close all;

W_bw  = 10e6;           % pass-band bandwidth 2W = 10 MHz
Ts    = 1 / W_bw;       % sampling period = 1/(2W) = 0.1 us
```

```
NFFT  = 2048;           % FFT size
fs    = W_bw;           % sampling frequency = 10 MHz

%% ---- PDP Definition ----
gain_dB  = [-2, 0, -1, -6, -9, -14];       % path gains in dB
delay_us = [0, 1.8, 3.5, 5.7, 8.1, 12.3]; % path delays in microseconds
L = length(gain_dB);

sigma2_lin = 10.^(gain_dB / 10);
sigma2 = sigma2_lin / sum(sigma2_lin);

% Ts = 0.1 us  →  tap_k = round(delay_us / (Ts*1e6))
tap_idx = round(delay_us / (Ts * 1e6));

rng(42);
colors = {'b', 'r', 'g'};

figure('Name', 'Q2: Frequency Response | H(f) |^2 | 3 Realizations');
hold on;

for run = 1:3

    h = zeros(1, NFFT);      % pre-allocate

    for i = 1:L
        % Each path gain a_i ~ CN(0, sigma^2_i)
        % Real and imag parts each ~ N(0, sigma^2_i / 2)
        std_each = sqrt(sigma2(i) / 2);
        a_i = std_each * (randn + 1j * randn);
        h(tap_idx(i) + 1) = a_i;   % +1 for 1-based MATLAB indexing
    end

    H = fft(h, NFFT);
    f_MHz = (0:NFFT-1) * (fs/NFFT) / 1e6;   % 0 to fs in MHz
    plot(f_MHz, 10*log10(abs(H).^2), colors{run}, 'LineWidth', 1.2);

    end

%% ---- Coherence bandwidth ----
% Approx Bc ~ 1/tau_max = 1/12.3e-6 ~ 81 kHz
tau_mean = sum(sigma2 .* delay_us);
tau_sq_mean = sum(sigma2 .* (delay_us.^2));
tau_rms = sqrt(tau_sq_mean - tau_mean^2);

Bc_kHz = 1 / (5 * tau_rms * 1e-6) / 1e3;
Bc_MHz = Bc_kHz / 1e3;   % convert to MHz to match f_MHz axis

fhB_MHz = Bc_MHz : Bc_MHz : fs/1e6;
```

```
for k = 1:length(fhB_MHz)
    if k == 1
        xline(fhB_MHz(k), 'w--', 'LineWidth', 1.2, ...
              'DisplayName', sprintf('B_c \\approx %.0f kHz', Bc_kHz), ...
              'Interpreter', 'tex');
    else
        xline(fhB_MHz(k), 'w--', 'LineWidth', 1.2, 'HandleVisibility', 'off');
    end
end

xlabel('Frequency (MHz)', 'Interpreter', 'tex');
ylabel('10 log_{10} |{\itH}({\itf})|^2  (dB)', 'Interpreter', 'tex');
title({'Frequency Response of Fading Channel with Specified PDP using 3 Independent Channel Rea
legend('Realization 1', 'Realization 2', 'Realization 3', sprintf('B_c \\approx %.0f kHz', Bc_k
grid on;
xlim([0, fs/1e6]);
```

A.6 Q3_PDP.m

```
%% Problem 3: Time-Varying Frequency-Selective Fading | 3D Plot


clear; clc; close all;

%% ---- PDP Parameters  ----
W_bw     = 10e6;              % 2W = 10 MHz
Ts_tap   = 1/W_bw;            % tap spacing = 0.1 us
NFFT     = 2048;

gain_dB  = [-2, 0, -1, -6, -9, -14];
delay_us = [0, 1.8, 3.5, 5.7, 8.1, 12.3];
L        = length(gain_dB);


sigma2   = 10.^(gain_dB/10);
sigma2   = sigma2 / sum(sigma2);

tap_idx  = round(delay_us / (Ts_tap * 1e6));

%% ---- Doppler and Snapshot Parameters ----
fd        = 90;                     % max Doppler per path (Hz)
wd        = 2*pi*fd;
M         = 20;                     % SOS sinusoids per path (Zheng-Xiao)
t_snap_ms = 0:5:30;                 % [0 5 10 15 20 25 30] ms
t_snap    = t_snap_ms * 1e-3;       % in seconds
Nsnap     = length(t_snap);         % 7 snapshots
```

```
rng(42);
theta_all  = unifrnd(-pi, pi, 1, L);
phi_all    = unifrnd(-pi, pi, M, L);
varphi_all = unifrnd(-pi, pi, M, L);
n_vec      = (1:M)';

f_MHz = (0:NFFT-1) * (W_bw/NFFT) / 1e6;  % 0 to 10 MHz

% H_dB: (NFFT x Nsnap) matrix of |H(f, t)|^2 in dB
H_dB = zeros(NFFT, Nsnap);

for s = 1:Nsnap
    t_now  = t_snap(s);
    h_snap = zeros(1, NFFT);
    for i = 1:L
        alpha_n = (2*pi*n_vec - pi + theta_all(i)) / (4*M);

        Z_c = sqrt(2/M) * sum(cos(wd * t_now * cos(alpha_n) + phi_all(:,i)));
        Z_s = sqrt(2/M) * sum(cos(wd * t_now * sin(alpha_n) + varphi_all(:,i)));

        a_i = sqrt(sigma2(i)/2) * (Z_c + 1j*Z_s);

        h_snap(tap_idx(i) + 1) = h_snap(tap_idx(i) + 1) + a_i;
    end

    H_snap       = fft(h_snap, NFFT);
    H_dB(:, s)   = 10*log10(abs(H_snap).^2);
end

%% ---- 3D Surface Plot ----

[T_grid, F_grid] = meshgrid(t_snap_ms, f_MHz);

figure('Name', 'Q3: Time-Varying Frequency Response | 3D Surface');
surf(T_grid, F_grid, H_dB, 'EdgeColor', 'none');
colormap jet;
colorbar;
xlabel('Time  (ms)');
ylabel('Frequency  (MHz)');
zlabel('10 log_{10} |H(f,t)|^2  (dB)');
title({'Time-Varying Frequency Response of fading channel described by the given PDP and f_d =
view(45, 35);
grid on;

%% ---- 2D Overlay Plot ----
figure('Name', 'Q3: Frequency Snapshots Overlaid');
hold on;
cmap = lines(Nsnap);
```

```
for s = 1:Nsnap
    plot(f_MHz, H_dB(:,s), 'Color', cmap(s,:), 'LineWidth', 1.2);
end
xlabel('Frequency  (MHz)');
ylabel('10 log_{10} |H(f,t)|^2  (dB)');
title({'Time-Varying Frequency Response of fading channel at each snapshot time with f_d = 90 H
legend_str = arrayfun(@(x) sprintf('t = %d ms', x), t_snap_ms, 'UniformOutput', false);
legend(legend_str, 'Location', 'best');
grid on;

%% ====================================================================
%% BELLO SYSTEM FUNCTIONS
%% ====================================================================
% Recompute storing complex matrices
h_all     = zeros(NFFT, Nsnap);   % h(tau, t): delay-time plane
H_complex = zeros(NFFT, Nsnap);   % H(f,   t): frequency-time plane

for s = 1:Nsnap
    t_now = t_snap(s);
    h_tmp = zeros(1, NFFT);
    for i = 1:L
        alpha_n = (2*pi*n_vec - pi + theta_all(i)) / (4*M);
        Z_c = sqrt(2/M) * sum(cos(wd * t_now * cos(alpha_n) + phi_all(:,i)));
        Z_s = sqrt(2/M) * sum(cos(wd * t_now * sin(alpha_n) + varphi_all(:,i)));
        a_i = sqrt(sigma2(i)/2) * (Z_c + 1j*Z_s);
        h_tmp(tap_idx(i)+1) = h_tmp(tap_idx(i)+1) + a_i;
    end
    h_all(:, s)     = h_tmp.';
    tmp             = fft(h_tmp, NFFT);
    H_complex(:, s) = tmp.';
end

%% ---- Derived Bello functions via FFT over time axis ----
% S(f, nu)  = FT_t{ H(f,t) } | each row of H_complex FFT'd over Nsnap points
S_fn = fftshift(fft(H_complex, Nsnap, 2), 2);

% B(tau,nu) = FT_t{ h(tau,t) } | each row of h_all FFT'd over Nsnap points
B_tn = fftshift(fft(h_all,     Nsnap, 2), 2);

tau_max_disp = max(tap_idx) + 5;
tau_us_disp  = (0:tau_max_disp) * Ts_tap * 1e6;  % microseconds
tau_rows     = 1 : tau_max_disp+1;

dt_snap = 5e-3;

%% ===== Bello Domain 1 | h(tau, t) =====
[T3, TAU3] = meshgrid(t_snap_ms, tau_us_disp);
h_dB3      = 10*log10(abs(h_all(tau_rows, :)).^2 + 1e-12);
```

```
figure('Name', 'Bello 1: h(tau,t) | Delay-Time Plane');
surf(T3, TAU3, h_dB3, 'EdgeColor', 'none');
colormap jet; colorbar;
xlabel('Time  t  (ms)');
ylabel('Delay  \tau  (\mus)');
zlabel('|h(\tau,t)|^2  (dB)');
title({'Bello Function 1:  Input Delay-Spread Function  h(\tau, t)'});
view(45, 35); grid on;

%% ===== Bello Domain 2 | H(f, t) =====
[T4, F4] = meshgrid(t_snap_ms, f_MHz);
H_dB4    = 10*log10(abs(H_complex).^2 + 1e-12);

figure('Name', 'Bello 2: H(f,t) | Frequency-Time Plane');
surf(T4, F4, H_dB4, 'EdgeColor', 'none');
colormap jet; colorbar;
xlabel('Time  t  (ms)');
ylabel('Frequency  f  (MHz)');
zlabel('|H(f,t)|^2  (dB)');
title({'Bello Function 2:  Time-Varying Transfer Function  H(f, t)'});
view(45, 35); grid on;

%% ===== Bello Domain 3 | S(f, nu) =====
[NU5, F5] = meshgrid(nu_Hz, f_MHz);
S_dB5     = 10*log10(abs(S_fn).^2 + 1e-12);

figure('Name', 'Bello 3: S(f,nu) | Doppler-Spread Function');
surf(NU5, F5, S_dB5, 'EdgeColor', 'none');
colormap jet; colorbar;
xlabel('Doppler  \nu  (Hz)');
ylabel('Frequency  f  (MHz)');
zlabel('|S(f,\nu)|^2  (dB)');
title({'Bello Function 3:  Doppler-Spread Function  S(f, \nu)'});
view(45, 35); grid on;

%% ===== Bello Domain 4 | B(tau, nu) =====
[NU6, TAU6] = meshgrid(nu_Hz, tau_us_disp);
B_dB6      = 10*log10(abs(B_tn(tau_rows, :)).^2 + 1e-12);

figure('Name', 'Bello 4: B(tau,nu) | Delay-Doppler Spread Function');
surf(NU6, TAU6, B_dB6, 'EdgeColor', 'none');
colormap jet; colorbar;
xlabel('Doppler  \nu  (Hz)');
ylabel('Delay  \tau  (\mus)');
zlabel('|B(\tau,\nu)|^2  (dB)');
title({'Bello Function 4:  Delay-Doppler Spread Function  B(\tau, \nu)'});
view(45, 35); grid on;
```